Universidad Internacional de La Rioja
Escuela Superior de Ingeniería y Tecnología

Máster Universitario en Ciberseguridad

Webscavul: Herramienta para el análisis y gestión de vulnerabilidades web

Trabajo fin de estudio presentado por:	Alberto González Martínez
Tipo de trabajo:	Desarrollo Software
Director/a:	Dr. Sergio Mauricio Martínez Monterrubio
Fecha:	10 de septiembre 2025

Resumen

El presente proyecto tiene como finalidad el desarrollo de Webscavul, una herramienta *full-stack* gratuita y de código abierto orientada a facilitar el análisis y gestión de vulnerabilidades web. Parte de una investigación sobre las principales herramientas de análisis de vulnerabilidades y de la necesidad de crear una solución gratuita, accesible, que no requiera que el usuario tenga amplios conocimientos técnicos sobre ciberseguridad, creando así un entorno que simplifique el despliegue seguro de aplicaciones. Se ha seguido una metodología de desarrollo ágil, diseñando los requisitos e interfaces de la aplicación e incorporando tecnologías modernas como Python y Next.js, con el fin de garantizar una solución actual y robusta. Los resultados muestran la obtención de una herramienta funcional, usable y que facilita el análisis y la gestión de vulnerabilidades, contribuyendo a reducir la barrera de entrada para la adopción de la seguridad en el desarrollo web o como complemento a otras herramientas.

Palabras clave: Webscavul, vulnerabilidades en web, Python, Next.js.

Abstract

The purpose of this project is to develop Webscavul, a free, open-source full-stack tool designed to facilitate the analysis and management of web vulnerabilities. It is based on research into the main vulnerabilities analysis tools and the need to create a free, accessible solution that does not require the user to have extensive technical knowledge of cybersecurity, thus creating an environment that simplifies the secure deployment of applications. An agile development methodology was followed, designing the application's requirements and interfaces and incorporating modern technologies such as Python and Next.js, guaranteeing a cutting edge and robust solution. The results show the creation of a functional, user-friendly tool that facilitates the analysis and management of vulnerabilities, helping to reduce the entry barrier for the adoption of security in web development or as a complement to other tools.

Keywords: Webscavul, web vulnerabilities, Python, Next.js.

Índice de contenidos

1. Capítulo 1: Introducción 1
1.1. Motivación 2
1.2. Planteamiento del problema 4
1.3. Estructura del trabajo 4
1.4. Hipótesis 5
1.4.1. Hipótesis Nula (H0) 5
1.4.2. Hipótesis Alternativa (HA) 5
1.5. Objetivos 6
1.5.1. Objetivo general 6
1.5.2. Objetivos específicos 6
2. Capítulo 2: Estado del arte 7
2.1. Introducción 7
2.2. Principales vulnerabilidades web 7
2.2.1. A01:2021-Pérdida de Control de Acceso 9
2.2.2. A02:2021-Fallos Criptográficos 9
2.2.3. A03:2021-Inyección 9
2.2.4. A04:2021-Diseño Inseguro 10
2.2.5. A05:2021-Configuración incorrecta de seguridad 10
2.2.6. A06:2021-Componentes Vulnerables y Obsoletos 10
2.2.7. A07:2021-Fallos de Identificación y Autenticación 11
2.2.8. A08:2021-Fallos de Integridad de Software y Datos 11
2.2.9. A09:2021-Fallos de Registro y Monitoreo de Seguridad 11
2.2.10. A10:2021-Falsificación de solicitud del lado del servidor 12
2.3. Herramientas de análisis de vulnerabilidades 12

2.3.1. OWASP ZAP 12
2.3.2. Burp Suite 13
2.3.3. Acunetix 13
2.3.4. Nikto 14
2.4. Limitaciones de las soluciones actuales 14
2.5. Conclusiones 15
3. Capítulo 3: Objetivos concretos y metodología de trabajo 16
3.1. Objetivo general 16
3.2. Objetivos específicos 16
3.3. Metodología del trabajo 17
4. Capítulo 4: Desarrollo de Webscavul 19
4.1. Tecnologías 19
4.1.1. Python 19
4.1.2. Next.js 22
4.2. Identificación de requisitos 23
4.2.1. Requisitos funcionales 23
4.2.2. Requisitos no funcionales 25
4.3. Diagrama de casos de uso 26
4.4. Desarrollo del API 27
4.4.1. Funcionamiento del API 28
4.4.2. Base de datos 32
4.5. Desarrollo del motor de escaneo y análisis 34
4.5.1. Análisis de cabeceras 34
4.5.2. Información general y certificado SSL/TLS 39
4.5.3. Análisis sintáctico del código HTML 41

4.6. Desarrollo de la interfaz de usuario 45
4.6.1. Estructura de la aplicación 46
4.6.2. Vistas de la aplicación 48
4.6.3. Integración con la API 58
4.6.4. Generación de contexto y soluciones con modelos de lenguaje 59
4.7. Despliegue de la aplicación 61
4.8. Pruebas y evaluación de la aplicación 65
4.8.1. Pruebas de usabilidad y experiencia de usuario 65
4.8.2. Pruebas de funcionamiento 68
5. Conclusiones 73
6. Trabajo futuro 76
Referencias bibliográficas 77
Anexo A. Encuesta de usabilidad y experiencia de usuario 80
Anexo B. Archivo HTML de pruebas 81

Índice de figuras

Figura 1. Gráfico evolutivo de número de usuarios en Internet. Fuente: (Statista, 2024) 1

Figura 2. Principales vulnerabilidades 2023-2024 Fuente: (ENISA, 2024) 8

Figura 3. Diagrama de casos de uso de la aplicación. Fuente: Elaboración propia.....26

Figura 4. Estructura de los archivos y carpetas de la API de la aplicación. Fuente: Elaboración propia.....28

Figura 5. Ejemplo de implementación de método GET con FastAPI. Fuente: Elaboración propia. 29

Figura 6. Ejemplo de implementación de método POST con FastAPI. Fuente: Elaboración propia.....30

Figura 7. Ejemplo de implementación de método PUT con FastAPI. Fuente: Elaboración propia. 30

Figura 8. Ejemplo de implementación de método DELETE con FastAPI. Fuente: Elaboración propia.....31

Figura 9. Ejemplo de cuerpo y respuesta para una llamada tipo POST usando Postman. Fuente: Elaboración propia.....31

Figura 10.Ejemplo de la respuesta para una llamada tipo GET usando Postman. Fuente: Elaboración propia.....32

Figura 11. Método para la eliminación de un informe. Fuente: Elaboración propia.....33

Figura 12. Método para la creación de un pool de hilos con conexión a la base de datos. Fuente: Elaboración propia.....33

Figura 13. Fragmento de código para la inicialización del objeto "session". Fuente: Elaboración propia.....35

Figura 14. Método para la ejecución de la petición HTTP sobre el dominio requerido. Fuente: Elaboración propia.....36

Figura 15. Método para el análisis de la cabecera "Referrer". Fuente: Elaboración propia....38

Figura 16. Método para la obtención de información general de un servidor web. Fuente: Elaboración propia.....40

Figura 17. Método para el análisis de la información SSL. Fuente: Elaboración propia. 40

Figura 18. Método para la carga del contenido de la página web requerida. Fuente: Elaboración propia.....42

Figura 19. Método para la extracción de la información del contenido de la página web. Fuente: Elaboración propia. 42

Figura 20. Método para la extracción de la información de los enlaces de una página. Fuente: Elaboración propia.....43

Figura 21. Código del objeto "Vulnerability". Fuente: Elaboración propia. 43

Figura 22. Método para el análisis de los enlaces de una página web. Fuente: Elaboración propia.....44

Figura 23. Fragmento de código con los métodos para el análisis y extracción de información sobre la página web. Fuente: Elaboración propia. 45

Figura 24. Estructura de los archivos y carpetas de la interfaz de la aplicación. Fuente: Elaboración propia.....46

Figura 25. Código del componente "InputDomain" de la aplicación. Fuente: Elaboración propia.....47

Figura 26. Página principal (Home) de Webscavul. Fuente: Elaboración propia. 48

Figura 27. Error mostrado por la aplicación tras introducir un dominio no válido. Fuente: Elaboración propia.....49

Figura 28. Mensaje mostrado mientras la aplicación realiza un escaneo. Fuente: Elaboración propia.....49

Figura 29. Error mostrado por la aplicación tras ocurrir un error en el escaneo. Fuente: Elaboración propia.....50

Figura 30. Página con el listado de informes de Webscavul. Fuente: Elaboración propia. 51

Figura 31. Menú desplegable con opción para eliminar el informe. Fuente: Elaboración propia. 52

Figura 32. Página con el detalle del informe de Webscavul. Fuente: Elaboración propia.....53

Figura 33. Listado con el resultado del análisis de las cabeceras realizado por Webscavul. Fuente: Elaboración propia. 54

Figura 34. Página con el tablero de vulnerabilidades de Webscavul. Fuente: Elaboración propia.....55

Figura 35. Modal para la creación de una nueva lista. Fuente: Elaboración propia. 55

Figura 36. Menú desplegable con la opción de eliminar y editar una lista. Fuente: Elaboración propia.....56

Figura 37. Modal con la información detallada de una vulnerabilidad. Fuente: Elaboración propia.....57

Figura 38. Menún desplegable con la opción de eliminar y mover una tarea. Fuente: Elaboración propia.....57

Figura 39. Ejemplo de solución y contexto generado por Webscavul usando modelos de lenguaje. Fuente: Elaboración propia. 58

Figura 40. Método para la ejecución de la llamada a la API que elimina un informe. Fuente: Elaboración propia.....59

Figura 41. Método que maneja la información recibida tras la eliminación de un informe. Fuente: Elaboración propia. 59

Figura 42. Fragmento de código para la creación y ejecución de las consultas al modelo de lenguaje. Fuente: Elaboración propia. 60

Figura 43. Método que maneja la respuesta recibida del modelo de lenguaje. Fuente: Elaboración propia.....61

Figura 44. Ejecución de comandos para el despliegue de la aplicación. Fuente: Elaboración propia.....62

Figura 45. Fragmento del archivo docker-compose.yml para el servicio de la API. Fuente: Elaboración propia.....63

Figura 46. Fragmento del archivo docker-compose.yml para el servicio de la interfaz. Fuente: Elaboración propia.....63

Figura 47. Fragmento del archivo docker-compose.yml para el servicio de la base de datos. Fuente: Elaboración propia. 63

Figura 48. Código del archivo Dockerfile de la API. Fuente: Elaboración propia. 64

Figura 49. Código del archivo Dockerfile de la interfaz. Fuente: Elaboración propia. 64

Índice de tablas

Tabla 1. Listado de requisitos funcionales de la aplicación. Fuente: Elaboración propia. 23

Tabla 2. Listado de requisitos no funcionales de la aplicación. Fuente: Elaboración propia... 25

Tabla 3. Resultado de la encuesta sobre la usabilidad y experiencia de usuario. Fuente: Elaboración propia 66

Tabla 4. Listado de vulnerabilidades encontradas por Webscavul en meta etiquetas. Fuente: Elaboración propia 68

Tabla 5. Listado de vulnerabilidades encontradas por Webscavul en etiquetas script. Fuente: Elaboración propia 69

Tabla 6. Listado de vulnerabilidades encontradas por Webscavul en etiquetas link. Fuente: Elaboración propia 69

Tabla 7. Listado de vulnerabilidades encontradas por Webscavul en formularios. Fuente: Elaboración propia 70

Tabla 8. Listado de vulnerabilidades encontradas por Webscavul en enlaces. Fuente: Elaboración propia 72

1. Capítulo 1: Introducción

Desde el surgimiento de la Web 2.0 sobre los años 2000, caracterizada por la gran interacción que brinda a las páginas webs, la manera en la que las personas interaccionan, consumen contenido, acceden a la información o realizan compras a través Internet, ha cambiado de manera radical.

Debido a esto, en las últimas décadas, el **número de usuarios que utilizan Internet ha sufrido un incremento** tal y como se muestra en la siguiente figura. El número de usuarios que utilizan Internet ha ido en aumento año tras año, llegando en 2024 a los 5.500 millones de usuarios activos, lo que supone casi un 70% de la población mundial.

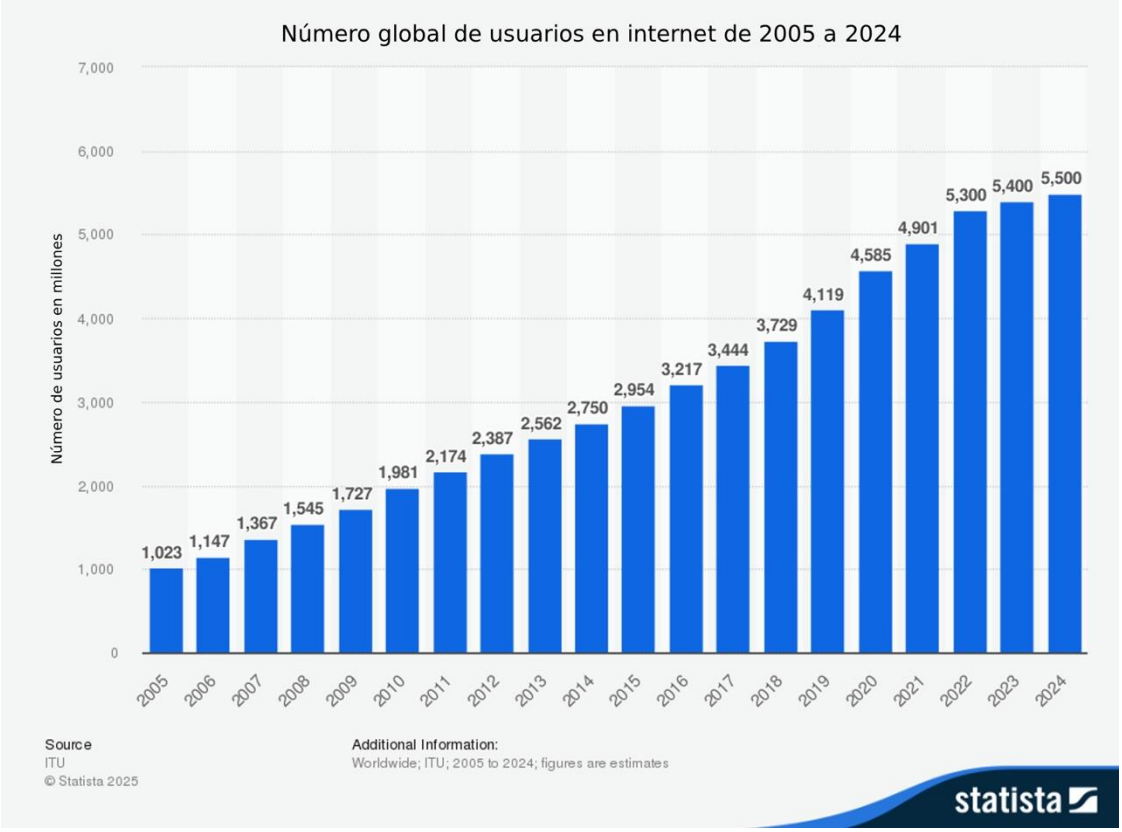


Figura 1. Gráfico evolutivo de número de usuarios en Internet.

Fuente: (Statista, 2024)

Asimismo, el **número de páginas webs** también **ha experimentado un gran crecimiento**, llegando a contabilizarse a marzo de 2025, 1.200 millones de dominios, de los cuales, cerca de 200 millones, son dominios activos (Netcraft, 2025).

Este gran incremento de usuarios y dominios activos, hacen que las aplicaciones o páginas webs, se hayan convertido en uno de los vectores de ataques más utilizados por los ciberdelincuentes hoy en día y, por lo tanto, uno de los elementos más críticos, debido a que estas páginas web, en muchos casos, suponen un importante flujo de ingresos para las empresas, y lo que es más importante, almacenan información confidencial de clientes y usuarios.

Es por esto por lo que, cualquier **brecha de seguridad** puede **causar** a la empresa **pérdidas económicas y daños a la reputación y/o continuidad del negocio**.

Las herramientas de análisis de vulnerabilidades en páginas y aplicaciones web ayudan a identificar y detectar posibles errores y brechas de seguridad antes de que estas se produzcan, lo que permite a las empresas, no solo a reducir pérdidas monetarias, si no también entregar un producto o servicio seguro, confiable y que proteja la integridad y confidencialidad de la información de clientes y usuarios.

1.1. Motivación

En un entorno cada vez más y más interconectado y digitalizado, el número de páginas y aplicaciones web crece de manera constante, así como el número de usuarios que acceden a estas (Statista, 2024), siendo estas páginas web, un mercado muy atractivo para los ciberdelincuentes, debido, en muchos casos, a la facilidad con la que se pueden realizar los ataques, a la falta de seguridad que presentas muchas aplicaciones y a la exposición que estos servicios tienen en Internet.

Las principales causas de esta falta de seguridad en la aplicaciones y páginas web, se debe a diferentes factores, como puede ser la falta de experiencia de los desarrolladores, la creciente complejidad de las tecnologías utilizadas o en muchos de los casos, la escasez de tiempo y recursos que se emplean en el ciclo de desarrollo de una aplicación.

Todo esto hace que, hoy en día, las **páginas web** sean uno de los **vectores de ataque más utilizados por los atacantes**, siendo, en este caso, el XSS (Cross-site Scripting) y la inyección SQL, las vulnerabilidades más importantes del pasado año 2024 según el informe de amenazas de ENISA (ENISA, 2024).

Esto pone de manifiesto la gran importancia y el reto que supone para las empresas securizar y detectar las vulnerabilidades que presentan sus páginas web y contar con herramientas que faciliten y ayuden en este trabajo es algo muy importante.

En este contexto, se propone el desarrollo de una herramienta de análisis de vulnerabilidades para aplicaciones y páginas web, orientado a realizar un **escaneo básico y automático** en modo **caja negra/gris**, este término se para expresar el nivel de conocimiento que se tiene sobre un sistema o aplicación.

Los términos caja negra y gris se utilizan en el ámbito de la ciberseguridad para expresar el nivel de conocimiento que se tiene sobre un sistema o aplicación. En este caso, se refiere a que la herramienta trabajaría sin ningún tipo de información sobre el sistema o con información limitada sobre este.

El **escaneo de la página** web se realizará mediante la **introducción de una dirección IP o URL**, que permita obtener información útil sin necesidad de tener conocimiento técnicos y exhaustivos de desarrollo web, así como organizar y gestionar los reportes y las posibles soluciones a las vulnerabilidades detectadas.

La elección de este tema responde tanto al interés personal en el ámbito de la ciberseguridad y el desarrollo web, campo en el que hasta el momento me estoy desarrollando de forma profesional, como a la necesidad de desarrollar una herramienta que sea gratuita, accesible y que permita a desarrolladores independientes, estudiantes, pequeñas empresas u otros interesados, mejorar la seguridad y la gestión de incidencias en aplicaciones web.

En mi caso personal, los profesionales que trabajan en el desarrollo de páginas web tienen conocimientos técnicos en este ámbito, por ejemplo, tecnologías actuales, terminología web…, pero en muchas ocasiones este conocimiento no se extiende ni se relaciona al ámbito de la seguridad.

Es por esto por lo que los resultados que proporciona una herramienta de análisis de vulnerabilidades, los **costes** o incluso la utilización o **aprendizaje** de estas herramientas, puede suponer una **barrera de entrada** que provoca que en muchos equipos de desarrollo **no se empleen este tipo de soluciones**.

1.2. Planteamiento del problema

En la actualidad, el creciente número de usuario en Internet, junto con la gran cantidad de sitios webs que están expuestos, ha convertido a estos en uno de los principales objetivos de ataques.

La mayoría de las **pequeñas y medianas empresas o desarrolladores, no tienen los recursos** adecuados, tiempo o conocimientos técnicos para evaluar de una forma correcta la seguridad de sus aplicaciones web. Esta falta de recursos y conocimientos puede provocar que los fallos de seguridad que tienen estas páginas webs sean aprovechados por los ciberdelincuentes, resultando en pérdidas económicas, interrupciones del servicio o que los datos sensibles de los usuarios se vean comprometidos.

Por ello, a pesar de que hoy en día existan **soluciones comerciales** robustas para realizar el análisis de vulnerabilidades en entornos web, muchas de ellas **imponen algunas barreras**, como pueden ser la complejidad, el alto costo o la necesidad de conocimientos técnicos avanzados para su uso de forma eficiente.

Esto plantea un **problema** claro y es la necesidad de contar con herramientas simples, accesibles y automatizadas que ofrezcan la capacidad de detectar los fallos y errores básicos de seguridad en una aplicación web, así como que ofrezcan soluciones fáciles a los errores detectados.

Para hacer frente a esta problemática, se propone **desarrollar una herramienta** propia, de tipo **caja negra/gris**, que permita realizar un **análisis de vulnerabilidades de forma automática** a partir de la **URL o dirección IP del servidor**, generando a partir de esta un informe con información sobre la página web introducida, así como la creación de un tablero de gestión de los fallos encontrados.

1.3. Estructura del trabajo

En el **primer capítulo**, se describe el problema planteado, así como la motivación de este trabajo y se definen los objetivos generales y específicos que se esperan cumplir en el desarrollo de la solución planteada.

En el **segundo capítulo**, se hace un estudio e investigación de las principales herramientas presentes actualmente en el mercado, así como un resumen de las principales vulnerabilidades presentes en el desarrollo web.

En el **tercer capítulo**, se definen los objetivos que guían el desarrollo de la herramienta y se expone la metodología usada, en cada una de las fases de desarrollo, para alcanzar dichos objetivos.

En el **cuatro capítulo**, se detallará el proceso de desarrollo de la herramienta, Webscavul, abordando las diferentes decisiones de diseño y arquitectura tomadas. El capítulo inicia con la explicación de las tecnologías y lenguajes usados, tras esto, se definen los requisitos, tanto funcionales como no funcionales de la aplicación y se presenta el diagrama de casos de uso. A continuación, se explica de forma detallada el desarrollo de cada una de las partes principales que componen la aplicación como son, la API, el motor de escaneo y análisis y la interfaz de usuario y finalmente se presentan los resultados de las pruebas de experiencia de usuario y usabilidad y funcionamiento.

En los **últimos capítulos**, se exponen cuáles han sido las conclusiones obtenidas tras el desarrollo del trabajo y las líneas de trabajo futuro.

1.4. Hipótesis

1.4.1. Hipótesis Nula (H0)

El uso de herramientas de detección de vulnerabilidades y errores en páginas web no mejora la seguridad de una aplicación en comparación a un escaneo manual y es necesario tener conocimientos técnicos para poder utilizar una herramienta destinada a ese fin.

1.4.2. Hipótesis Alternativa (HA)

Las herramientas de detección de vulnerabilidades y errores en páginas web ayudan a mejorar significativamente la seguridad de una aplicación frente a un escaneo manual básico y, además, es posible el uso de estas sin la necesidad de tener conocimientos técnicos avanzados.

1.5. Objetivos

1.5.1. Objetivo general

Realizar un **análisis y estudio de las principales vulnerabilidades** en páginas y aplicaciones web y **desarrollar una herramienta** que permita, a partir de una URL o dominio, detectar las vulnerabilidades o brechas de seguridad existentes de forma automática, gratuita y accesible contribuyendo de esta forma a la mejora de la seguridad en entornos web, creando así una opción complementaria a otras herramientas que se encuentran en el mercado en la actualidad y proporcionando una opción amigable y fácil de usar para equipos que quieran mejorar la seguridad de sus aplicaciones.

1.5.2. Objetivos específicos

Los **objetivos específicos** de este trabajo son los siguientes:

1. Investigación de las principales vulnerabilidades presentes en las aplicaciones web modernas, siguiendo estándares como OWASP Top Ten (Foundation, 2021).
2. Análisis del funcionamiento de las herramientas de escaneo web actuales.
3. Diseño y desarrollo de una herramienta propia que permita de forma automatizada y con una interfaz amigable, detectar las vulnerabilidades y amenazas básicas presentes en una aplicación.
4. Investigación del funcionamiento e implementación de modelos de lenguaje natural que permitan generar reportes menos técnicos y más accesibles, con soluciones eficaces y claras.
5. Evaluación de la eficacia de la herramienta desarrollada utilizando entornos de pruebas controlados, comparando el resultado con escaneaos manuales.

2. Capítulo 2: Estado del arte

2.1. Introducción

En este apartado se explicará el contexto en el que se desarrolla la aplicación, así como el panorama actual en lo que se refiere al desarrollo web y el análisis de vulnerabilidades.

En primer lugar, se realizará un análisis y explicación de las principales vulnerabilidades y amenazas encontradas en los entornos web y sus posibles impactos.

A continuación, se realizará una comparación y análisis de las herramientas de análisis de vulnerabilidades web y de las plataformas de gestión de vulnerabilidades más conocidas y actuales del mercado, para así obtener una visión global de las capacidades, funcionalidades, limitaciones y características de estas.

Tras haber analizado el estado general de las herramientas disponibles, se expondrán las limitaciones que estas poseen, así como las posibles oportunidades de mejora y las características y funcionalidades que serán susceptibles de ser implementadas a la hora de desarrollar la herramienta.

2.2. Principales vulnerabilidades web

Como se ha podido observar en el **reporte de amenazas de 2024 realizado por ENISA**, (ENISA, 2024), algunas de las **principales vulnerabilidades** expuestas en este reporte **son las relacionadas con páginas web**. De hecho, como se muestra en la siguiente figura, las vulnerabilidades relacionadas o las que pueden ser explotadas en páginas web, conforman un gran porcentaje de las amenazas detectadas.

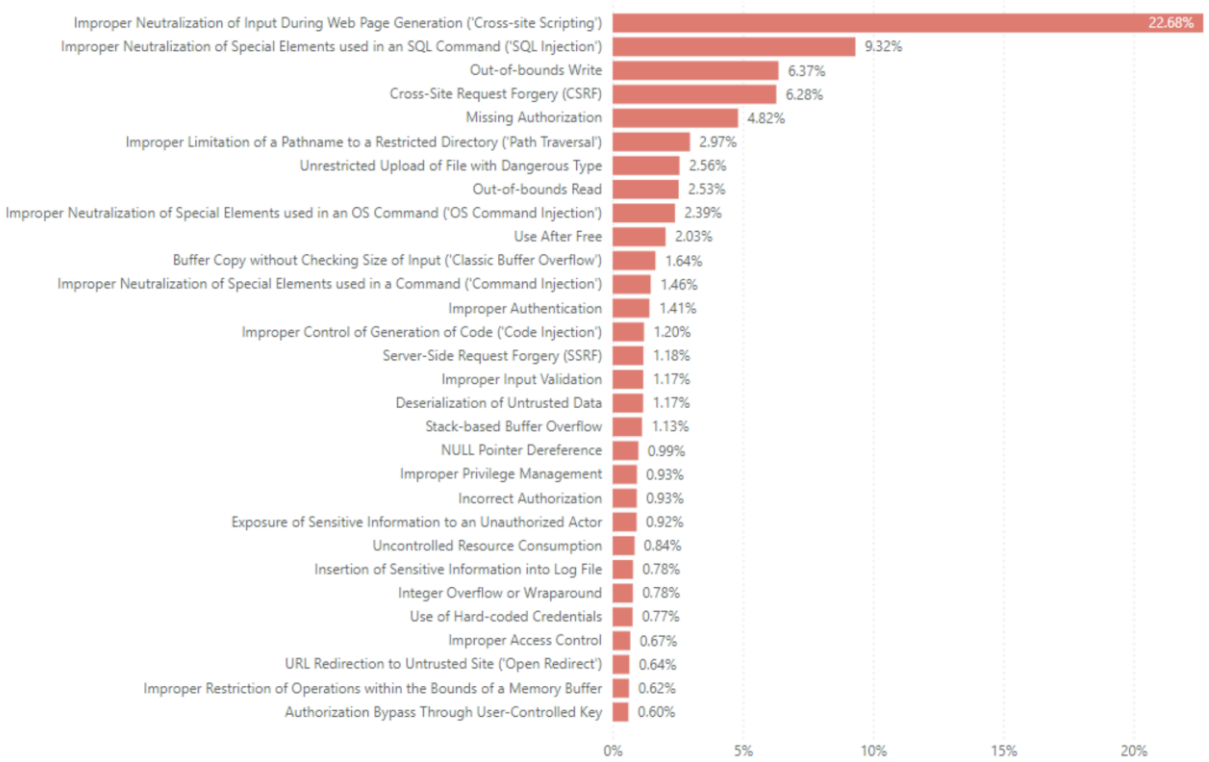


Figura 2. Principales vulnerabilidades 2023-2024

Fuente: (ENISA, 2024)

Uno de los puntos más importantes y básicos a la hora de estudiar y analizar el estado actual de los entornos web, es tener el conocimiento de las principales vulnerabilidades web existentes, para así poder enfocar y priorizar el desarrollo y el estudio en aquellas que tienen un impacto y prevalencia en un entorno real, y que representan riesgos significativos y con un mayor grado de incidencia sobre las dimensiones de la seguridad en las aplicaciones.

Para este fin, uno de los recursos más reconocidos y utilizados a nivel internacional es el **proyecto OWASP (*Open Worldwide Application Security Project*),** es una fundación sin ánimo de lucro que se dedica a mejorar la seguridad en las aplicaciones web, siendo uno de los principios fundamentales de esta que todos los materiales estén disponibles de forma gratuita y accesible desde su página web, uno de los proyectos más conocidos de esta fundación el conocido como OWASP Top 10 (Cloudflare, s.f.).

OWASP Top 10, es un documento que se encarga de recopilar las vulnerabilidades y amenazas más importantes o críticas con el fin de concienciar a las desarrolladores y empresas sobre la seguridad de las aplicaciones web. Fue publicado en el año 2021 y contempla las siguientes vulnerabilidades (Foundation, 2021):

2.2.1. A01:2021-Pérdida de Control de Acceso

El **control de acceso** es una de las partes más importantes de una aplicación ya que garantiza que los usuarios de la aplicación accedan y realicen acciones según el nivel de privilegios o rol que tengan. Esta pérdida o evasión del control de acceso puede conllevar graves riesgos para la seguridad de la aplicación, afectando a la confidencialidad de los datos y recursos, debido a que los atacantes podrían acceder a información no autorizada o realizar acciones no permitidas.

Algunos ejemplos de vulnerabilidades que explotan la pérdida de control de acceso son Referencias Directas Inseguras a Objetos (IDOR), Elevación de Privilegios, y Evasión de las Comprobaciones de Acceso.

2.2.2. A02:2021-Fallos Criptográficos

La **criptografía y los algoritmos criptográficos** en aplicaciones web son usados para guardar la confidencialidad y evitar que los sistemas y datos sensibles sean comprometidos. Por ello, el uso de algoritmos débiles, como por ejemplo MD5 o RC4, la falta de cifrado en las comunicaciones o en los datos almacenados y la mala gestión de las claves, exponiendo estas en el código o usando claves predeterminadas o simples, puede suponer una gran brecha de seguridad para las aplicaciones web.

2.2.3. A03:2021-Inyección

Las **vulnerabilidades que explotan la inyección** son uno de los ataques más comunes en los entornos web, debido en muchas ocasiones, a la facilidad de explotación de estos. Este tipo de fallos, se deben, en la mayoría de las ocasiones a una falta de validación y sanitización de las entradas de los usuarios y permiten a los atacantes ejecutar comandos, realizar operaciones dentro de una base de datos, almacenar información maliciosa para obtener información de otros usuarios...

Algunas de las vulnerabilidades relacionadas con la inyección son*, Cross-site Scripting* (XSS), *SQL Injection, Code injections, Lightweight Directory Application Protocol (LDAP) o XPATH injection* (Bach-Nutman, 2020).

2.2.4. A04:2021-Diseño Inseguro

La categoría de **diseño inseguro** engloba todas aquellas prácticas dentro del ciclo de vida de desarrollo software que no tienen en cuenta los principios de seguridad en las fases iniciales de diseño de una aplicación. Esta categoría en sí no es una vulnerabilidad o vector de ataque que pueda ser explotado, pero sí que es la causa principal de muchas de las vulnerabilidades presentes en esta lista.

Algunos ejemplos de prácticas inseguras dentro del desarrollo software son, falta de validación de los requisitos de seguridad, no realizar controles o pruebas del software o no aplicar el principio de mínimo privilegio.

2.2.5. A05:2021-Configuración incorrecta de seguridad

En esta categoría se encuentran los errores o vulnerabilidades que son debidos a **fallos en la configuración de la aplicación o componentes de esta**, como pueden ser, por ejemplo, servidores, bases de datos, aplicaciones, *plugins*, cabeceras... Estas configuraciones incorrectas de seguridad es una de las causas más comunes de filtrado de datos y puede ser explotada fácilmente por los atacantes si no se aplica, lo que se conoce como prácticas de endurecimiento (*hardening*), que tiene como finalidad pasar controles sobre los distintos componentes de una aplicación para reducir al máximo la superficie de ataque de estos.

Algunos ejemplos de configuraciones incorrectas de seguridad son, uso de valores o contraseñas por defecto, exposición de funcionalidades innecesarias como el listado de directorios de un servidor web o no incluir cabeceras de seguridad como por ejemplo "HttpOnly" o "Secure".

2.2.6. A06:2021-Componentes Vulnerables y Obsoletos

Las aplicaciones web actuales suelen contar con software, dependencias, librerías o *plugins*, los cuales son usados en el desarrollo para ahorrar tiempo o implementar funcionalidades. Esto es una gran ventaja y permite construir aplicaciones mejores y más robustas en menos tiempo, pero puede ser un grave riesgo cuando estos componentes no se actualizan, dando lugar a brechas de seguridad debido a vulnerabilidades conocidas en versiones específicas.

Por ello, es importante que se mantengan actualizadas los componentes utilizados en una aplicación y eliminar aquellos que no son necesarios o no se utilizan.

2.2.7. A07:2021-Fallos de Identificación y Autenticación

Una de las partes más importantes de una aplicación web, si existe, es la **identificación y autenticación de los usuarios** que acceden a ella, esto se debe a que es el mecanismo encargado de controlar los permisos, accesos y roles, por lo que, si este mecanismo es vulnerado, los atacantes podrían acceder a distintas funcionalidades y datos que pueden ser sensibles, confidenciales y/o críticos.

Se encuentra en el puesto siete de la lista debido a que hay numerosos *frameworks* estandarizados que permiten la implementación de mecanismo de control de acceso robustos.

2.2.8. A08:2021-Fallos de Integridad de Software y Datos

Esta categoría hacía referencia a las situaciones en las que una aplicación web utiliza o confía en datos, software o código que no es propio, es decir, se centra en la fase de suministro y actualización de los componentes y/o software.

Los riesgos que engloba esta categoría son, por ejemplo, la carga y ejecución de código sin verificación previa desde fuentes no confiables, la falta de verificación de la integridad de las actualizaciones o el uso de librerías, archivos de configuración o *plugins* de terceros.

2.2.9. A09:2021-Fallos de Registro y Monitoreo de Seguridad

Los fallos de registro y monitoreo de seguridad ocurren cuando una aplicación web no implementa de forma correcta mecanismos que registren los eventos relevantes que puedan ser útiles para detectar, alertar y responder ante actividades maliciosas o anómalas.

El registro de fallos es un área muy desafiante debido a que es complicado elaborar test o pruebas para comprobar la eficacia de los mecanismos implementados y una implementación

incorrecta o insuficiente puede tener un impacto en la detección, alerta y visibilidad de los incidentes y el estudio forense.

2.2.10. A10:2021-Falsificación de solicitud del lado del servidor

La Falsificación de solicitud del lado del servidor o SSRF (*Server Side Request Forgery*), es una vulnerabilidad añadida a la lista de OWASP Top 10, ya que es considerada por la comunidad como la más importante o crítica.

Esta vulnerabilidad puede suponer una gran amenaza para las aplicaciones web, ya que una explotación exitosa de esta permite, manipular las solicitudes HTTP realizadas por el servidor para conseguir que realice peticiones con otros fines, como pueden ser, acceso a datos o recursos sensibles o confidenciales o manipular y alterar datos o servicios presentes en la aplicación.

Fuentes: (Zamora, 2024) y (Foundation, 2021)

2.3. Herramientas de análisis de vulnerabilidades

En esta sección se realizará un análisis de las principales herramientas actuales que forman parte del ecosistema de soluciones para el análisis de vulnerabilidades, dando contexto e información sobre cada una de ellas e información comparativa entre estas como tipo de análisis que realizan, sistemas para los que son compatibles, dificultad en su uso, experiencia de usuario y compresión de la herramienta, tipo de licencia y en que escenarios suelen utilizarse.

Las herramientas que se analizarán en este apartado son OWASP ZAP, Burp Suite, Acunetix y Nikto (Swetha B., 2024).

2.3.1. OWASP ZAP

OWASP ZAP es **gratuita y de código abierto** desarrollada por la fundación OWASP y está orientada al análisis de vulnerabilidades en aplicaciones web. Esta herramienta funciona como un **proxy que intercepta el tráfico** entre la aplicación objetivo y el navegador del cliente con

el fin de analizarlo **(análisis pasivo)** o manipularlo **(análisis activo)**, pudiendo así detectar de forma automática diferentes tipos de vulnerabilidades.

Además, ofrece una **interfaz gráfica** para realizar los escaneos, un API REST que permite automatizar e integrar los escaneos dentro del ciclo de vida del desarrollo de la aplicación y otras funcionalidades como soporte para *plugins*, módulo para realizar *fuzzing o crawlers* para descubrir rutas dinámicas en la aplicación.

Por último, en lo que se refiere a fiabilidad y eficacia, las herramientas gratuitas y/o de código abierto como OWASP ZAP pueden ser igual de efectivos que otras soluciones de pago (Richard Amankwah, 2020), o incluso superando la efectividad del análisis dependiendo del modo de escaneo y el escenario presentado (Muzun Althunayyan, 2022).

2.3.2. Burp Suite

Burp Suite es una herramienta, que cuenta con una **versión gratuita y una de pago**, muy popular para el análisis de vulnerabilidades y auditorías de seguridad web a nivel profesional, desarrollada por PortSwigger. Al igual que OWASP ZAP, esta herramienta funciona mediante un **proxy que intercepta las peticiones** entre el servidor y el navegador del cliente.

Además de esta funcionalidad, Burp Suite implementa otras funcionalidades como un escáner automático de vulnerabilidades (*Scanner*), ataques de fuerza bruta con *payloads* personalizados (*Intruder*), reenvío de peticiones (*Repeater*), codificación y decodificación automática o manual (*Decoder*), comparación de las diferentes respuestas (*Comparer*) y más funcionalidades (PortSwigger, 2025).

Burp Suite es una herramienta **ampliamente usada en entornos profesionales** y con una gran precisión en el análisis y detección de vulnerabilidades.

2.3.3. Acunetix

Acunetix es una **herramienta comercial**, desarrollada por Invicti, que está especializada en el **análisis** automático de vulnerabilidades en **aplicaciones web y APIs**.

Su funcionamiento es como el de un **escáner dinámico de caja negra**, lo combina con técnicas de *crawling* o rastreo para encontrar todas las rutas posibles y tras esto realiza tanto

comprobaciones heurísticas, usando *payloads* para realizar ataques XSS o de inyección como comprobaciones por firmas, para detectar vulnerabilidades conocidas en diferentes partes de la aplicación.

De las herramientas analizadas **destaca por el alto coste de su licencia**, por lo que está orientada y pensada sobre todo para entornos corporativos, destaca también su buena interfaz y gestor de vulnerabilidades.

En las comparaciones con otras herramientas, Acunetix logra detectar **un gran porcentaje de vulnerabilidades** siendo una de las opciones **con escaneos más profundo y específicos**, sin tener en cuenta el coste. (Jahanzeb Shahid, 2022)

2.3.4. Nikto

Nikto es una **herramienta gratuita de código abierto** que está diseñada para realizar escaneos activos a servidores web.

No tiene interfaz gráfica ya que el manejo de la herramienta se realiza por consola (CLI) y su caso de uso suele ser para realizar auditorías rápidas y escaneos de servidores, ya que Nikto puede detectar archivos inseguros, configuraciones peligrosas, versiones de software obsoletas, cabeceras mal configuradas o directorios accesibles.

Es una herramienta que es ampliamente usada debido a su **rapidez en el escaneo**, **simplicidad** de uso y el **bajo consumo de recursos**, pero debido a que no realiza verificación activa de las vulnerabilidades detectadas ni pruebas de confirmación, puede generar más falsos positivos que otras herramientas.

2.4. Limitaciones de las soluciones actuales

Tras analizar las funcionalidades y características de algunas de las herramientas de análisis de vulnerabilidades más utilizadas actualmente, como OWASP ZAP, Burp Suite, Acunetix y Nikto, se pueden destacar algunas limitaciones o problemáticas:

1. **Curva de aprendizaje pronunciada**, muchas de las herramientas actuales, presentan una complejidad técnica elevada, esto provoca que sea más **costoso y difícil realizar análisis** de vulnerabilidades en proyectos con equipos de desarrollo más reducidos y

para particulares o que estos análisis se hagan de forma menos frecuente que lo que deberían. Según el reporte 2023 State of Open Source Security de Snyk, "el 80% de las empresas publican código de forma diaria o semanal pero solo el 27% auditan de manera continua" (Snyk, 2023).

2. **Interfaz de usuario compleja**, una interfaz difícil de comprender por el usuario hace que sea **más fácil cometer errores humanos** a la hora de realizar el análisis, interpretar los resultados obtenidos o no se use la herramienta de la forma o para el fin que fue diseñada (Cavallo, 2024).
3. **Falta de integración con gestores de proyectos**, la falta de integración nativa con gestores de proyectos o la carencia de mecanismos de gestión dentro de las herramientas de análisis, **dificulta la trazabilidad, asignación y seguimientos** de las tareas de corrección de las vulnerabilidades.
4. **Resultados difíciles de interpretar**, en muchas ocasiones, los informes generados por las herramientas de análisis no presentan la información de una forma clara, accesible para los usuarios o estructurada, esto puede provocar que no se solucionen las vulnerabilidades de una forma eficiente o **no se priorice de la forma correcta**. Por ello es muy importante, tener en cuenta los puntos clave propuestos en el artículo de Jason Firch en PurpleSec (Firch, 2024).
5. **Falta de integración con flujos de desarrollo**, la combinación de herramientas de análisis de vulnerabilidades con mecanismos de integración y entrega continua (CI/CD), hace que la tarea de despliegue y puesta en producción sea mucho más rápida, eficiente y ayuda a que las aplicaciones sean lo más robusta y seguras posibles.
6. **Ausencia de integración con modelos de lenguaje (LLMs)**, contar con la ayuda de modelos de lenguaje que generen explicaciones compresibles, posibles soluciones y pasos a seguir para la corrección de los fallos y vulnerabilidades detectadas hace que sea accesibles para usuarios con un conocimiento menos técnica y contribuye a que la **toma de decisiones sea mucho más eficiente**.

2.5. Conclusiones

El análisis de vulnerabilidades de aplicaciones web es una parte esencial en el ciclo de vida del desarrollo seguro de software. Sin embargo, durante el estudio del estado del arte se ha

revelado que las **herramientas actuales presentan algunas limitaciones** que suponen una barrera para que estas sean usadas de forma habitual por los equipos de desarrollo, tal y como demuestran algunas métricas plasmadas en el reporte elaborado por Snyk (Snyk, 2023).

Entre las **principales barreras identificadas** se encuentra la curva de aprendizaje pronunciada, la complejidad de las interfaces, falta de soluciones nativas para la gestión de las incidencias y vulnerabilidades encontradas, la dificultad para interpretar los resultados, la falta de integración con flujos CI/CD y la ausencia de modelos de lenguaje que hagan más accesible y entendible la tarea de análisis y remediación de las vulnerabilidades.

Estas carencias y limitaciones encontradas, además de otras no mencionadas como el coste asociado a las licencias de las herramientas comerciales, ponen de manifiesto la necesidad de **desarrollar un solución o herramienta gratuita, de código abierto**, **accesible** para todo tipo de usuarios interesados en **desarrollar aplicaciones web seguras**, que permita a los usuarios gestionar de una forma fácil e eficiente incidencias y vulnerabilidades encontradas y que se adapte a las necesidades de los equipos de desarrollo.

3. Capítulo 3: Objetivos concretos y metodología de trabajo

3.1. Objetivo general

El **objetivo general** de este trabajo es desarrollar una aplicación web que permita detectar vulnerabilidades o fallos presentes en las páginas web, con una interfaz sencilla de usar y que incluya una herramienta nativa para la gestión de los fallos o vulnerabilidades encontradas.

3.2. Objetivos específicos

Dentro de los **objetivos específicos** de este trabajo se encuentran:

1. Estudiar y comprender las vulnerabilidades que están presentes en el desarrollo web, así como sus posibles consecuencias y soluciones.
2. Investigar las herramientas y soluciones actuales.
3. Diseñar una interfaz sencilla y amigable que ayude a los usuarios a realizar el proceso de análisis y comprender los fallos encontrados.

4. Analizar los valores de las cabeceras de la respuesta del servidor web, dando consejos y valores recomendados.
5. Realizar un análisis del código HTML de la página web, para encontrar vulnerabilidades, malas configuraciones y recomendaciones que ayuden a mejorar la seguridad.
6. Desarrollar un planificador de proyectos que ayude a gestionar las vulnerabilidades y fallos encontrados durante el análisis.
7. Integrar el resultado del análisis con modelos de lenguaje que ayuden a aportar más información y contexto de los fallos y vulnerabilidades encontradas.

3.3. Metodología del trabajo

Para alcanzar los objetivos del trabajo, se ha adoptado una **metodología de trabajo ágil de tipo Kanban**. Este enfoque, a diferencia de otros como las metodologías de tipo Scrum, se centra más en el flujo de trabajo continuo, la continua entrega de tareas y la visualización de las tareas por medio de prioridades y estados.

Esta elección se debe a la experiencia previa con este tipo de metodología y se ha creído la mejor opción para proyectos que consisten en un desarrollo de software realizado por equipos pequeños, en este caso de una persona y que permite más **flexibilidad a la hora de ajustar las funcionalidades y tareas** por hacer sin necesidad de tener plazos de entrega fijos (FUENTES DEL BURGO & SEBASTIÁN PÉREZ, 2022).

El funcionamiento principal de este tipo de metodología es la **división de las funcionalidades** o requisitos de la aplicación en **tareas más pequeñas**, que se ordenan y dividen, de forma visual, en listas, que por lo general representan los estados que tienen estas tareas, como pueden ser, “Por hacer”, “En curso” y “Terminado” y el conjunto de estas listas y tareas comprende lo que se denomina tablero.

El desarrollo de la aplicación e investigación fue **dividido en diferentes fases o etapas** cada una de ellas con un tablero diferente, para separar y organizar todos los objetivos a llevar a cabo. Las fases son las siguientes:

1. **Investigación y planificación**, en esta fase inicial se realizó la investigación de las vulnerabilidades presentes en las aplicaciones web, sus soluciones, las herramientas o productos presentes en el mercado para detectarlas y gestionarlas, información sobre

las cabeceras HTTP y las tecnología o lenguajes que mejor se pueden adaptar al desarrollo de la aplicación, según las necesidades del trabajo y las prestaciones de estos.

2. **Diseño y arquitectura**, en esta fase se dividió el diseño de la aplicación en las diferentes página o vistas de esta, cada una de ellas comprendiendo una tarea, teniendo en mente la usabilidad y simplicidad del diseño y se eligió la arquitectura y lenguajes utilizados en el desarrollo tanto de la parte *front-end* de la aplicación como de la parte *back-end* de esta.
3. **Desarrollo de la aplicación**, esta fase fue en la que más tareas se crearon dentro del tablero, debido a que tiene la mayor cantidad más funcionalidades y objetivos a realizar. Se separaron las tareas que comprenden la parte del desarrollo del *back-end* de la aplicación o API, de las del *front-end*.
4. **Pruebas y despliegue**, en esta fase final se han diseñado diferentes pruebas, tanto en el desarrollo, para probar que funcionan de forma correcta cada una de las tareas completadas del tablero, como para probar que la aplicación final, es capaz de detectar y notificar todas las vulnerabilidades o fallos de configuración para los que ha sido diseñada.

4. Capítulo 4: Desarrollo de Webscavul

El desarrollo de la aplicación Webscavul comprende el proceso de ingeniería de software completo, desde la **investigación de las diferentes tecnologías** disponibles, pasando por el diseño de **la base de datos**, **arquitectura de la aplicación**, captación de **requisitos** y **codificación** de las diferentes funcionalidades y vista.

En el transcurso de este apartado se explicará y desglosará el proceso seguido, así como los diferentes componentes, funcionamiento y partes más importantes de la aplicación.

4.1. Tecnologías

Una de las decisiones más importantes a la hora de comenzar un desarrollo, es elegir bien las tecnologías y lenguajes con los que se va a construir el proyecto, ya que es esencial conocer las prestaciones y limitaciones de cada uno de estos para lograr los objetivos y funcionalidades propuestos en la fase de diseño.

La aplicación tiene **dos partes o elementos principales**, por un lado, el ***back-end* o API** de la aplicación, que compone toda la lógica que realiza los escaneos y las operaciones en la base de datos, por otro lado, el ***front-end* o interfaz gráfica** de la aplicación.

4.1.1. Python

Para el **desarrollo de la API** de la aplicación se ha optado por utilizar el lenguaje de programación **Python**, el cual es conocido por su simplicidad y gran versatilidad.

En este proyecto Python es una opción ideal para construir una aplicación de escaneo de vulnerabilidades por varias razones:

1. **Amplia disponibilidad de librerías**: la comunidad de Python ha desarrollado numerosas librerías que simplifican muchas tareas complejas y hacen que, implementar algunas funcionalidades como realizar peticiones HTTP, analizar certificados SSL o analizar el HTML sean mucho más rápidas y eficientes.
2. **Facilidad de uso y legibilidad**: Python cuenta con una sintaxis amigable, clara y legible, la cual facilita el mantenimiento de código a largo plazo, en este caso, es algo al tener

en cuenta ya que se pretende seguir añadiendo funcionalidades al proyecto en el futuro, véase el apartado de trabajo futuro (6).

3. **Integración con bases de datos**: otro punto esencial, es la integración que tiene con la base de datos utilizada en este proyecto, MariaDB, este cuenta con una librería que incorpora constructores, pool de hilos para implementar concurrencia y las consultas parametrizadas, las cuales permiten que las operaciones que se realizan en la base de datos sean más seguras (Corporation, 2020).
4. **Lenguaje y funcionalidades gratuitos**.

Como se ha indicado previamente, Python tiene una **amplia variedad de librerías** creadas por la comunidad que hacen que sea mucho más sencillo la integración de diferentes funcionalidades en la aplicación. Para el **desarrollo del proyecto** se han usado las **siguientes librerías**.

4.1.1.1. FastAPI

Es una librería **moderna y rápida con alta eficiencia** para construir APIs usando Python, se caracteriza por ser intuitiva, rápida y fácil de programar, robusta y que ayuda a escribir un código mucho más limpio y mantenible (FastAPI, s.f.).

Además, cuenta con mecanismos para **manejar errores**, devolver diferentes **mensajes** o **códigos HTTP** e implementa todos los métodos HTTP existentes como GET, PUT, POST, DELETE…

4.1.1.2. Requests

Es una librería fundamental para el funcionamiento de la aplicación, sirve para **enviar peticiones HTTP a los dominios o URLs** que el usuario introduzca en la aplicación (Reitz, 2025).

Ofrece un API de alto nivel que permite acceder a la información de la respuesta del servidor de forma sencilla y rápida, como, por ejemplo, el estado de la petición o cabeceras de la respuesta.

4.1.1.3. Playwright

Playwright es un framework de pruebas integral para aplicaciones web modernas, la funcionalidad principal es la de **emular de un navegador dentro de la aplicación** para así poder interactuar con las páginas y conseguir la información deseada de esta.

Es uno de los pilares de la aplicación desarrollada ya que permite **cargar el contenido HTML** de la página, **simulando ser un cliente** que accede a la página web, siendo posible cargar el contenido estático de esta página que usan la tecnología de renderizado del lado del cliente (Client-Side Rendering CSR) directamente desde cliente ejecutando código Javascript y renderizando el DOM, para extraer y analizar el contenido en busca de los patrones y errores que se han definido.

Es importante recalcar que, aunque para algunos escenarios se podría usar únicamente la libería Request (4.1.1.2), es **cada vez más común** en el desarrollo web usar **frameworks** o **tecnología** que se **renderizan y compilan del lado del cliente**, como las aplicaciones de una única página (*Single Page Applications* o SPAs) (Mukhamadiev, 2018). Su popularidad se debe al rendimiento y la experiencia de usuario que ofrecen para páginas con contenido dinámico. Estas tecnologías han demostrado ser una evolución clave en el desarrollo web, por lo que su uso es indispensable para dar una cobertura de escaneo completa y efectiva.

4.1.1.4. Beautiful Soup

Beautiful Soup es una librería de Python que sirve para **extraer los datos de archivos HTML y XML** y ofrece funcionalidad para navegar, buscar y modificar el árbol de análisis (Richardson, 2025).

En el proyecto esta librería es usada para **analizar en profundidad la salida** que devuelve la librería anteriormente citada **Playwright** (4.1.1.3), en busca de elementos clave dentro del código HTML de la página, como pueden ser *metatags*, enlaces, recursos externos, formularios…

4.1.1.5. pyOpenSSL

pyOpenSSL es una librería que engloba diferentes funcionalidades de bajo nivel para que la aplicación sea capaz de conseguir **información relevante sobre la configuración de los certificados SSL/TLS del servidor** (pyOpenSSL, 2025).

Con esta librería se consigue información, tal como, emisor del certificado, fecha de validez, versión de TLS, algoritmos de cifrado de TLS…

4.1.2. Next.js

Para el desarrollo de la interfaz gráfica de la aplicación se ha utilizado el **framework de ReactJS**, llamado Next.js, esta tecnología se coloca a la vanguardia y como **una de las más usadas en la actualidad** (Vercel, 2025). La elección tomada se basa en las siguientes características:

1. **Renderizado híbrido**: Next.js es una tecnología capaz de combinar el renderizado del lado de servidor (*Server-Side Rendering* o SSR) con el renderizado del lado de cliente (*Client-Side Rendering* o CSR), facilitando que se puedan **obtener las ventajas de ambos enfoques** a la hora de desarrollar la aplicación.
2. **Rutas dinámicas**: la capacidad que tiene Next.js de manejar rutas dinámicas de una forma sencilla, ha sido uno de los puntos más importantes a la hora de elegir una opción, ya que es algo esencial para el flujo de la aplicación. Con esta funcionalidad, crea rutas de forma dinámica, por cada uno de los informes generados por la herramienta.
3. **Experiencia de desarrollo y modularidad**: la **arquitectura basada en componentes** de Next.js, hace que el proyecto cuente con un código mucho más mantenible y entendible, ya que las interfaces diseñadas previamente, se descomponen en diferentes componentes que será reusado en otras partes de la aplicación, como, por ejemplo, un botón.
4. **Integración con hojas de estilos SASS**: Next.js tiene de forma nativa integración con la extensión de hojas de estilos SASS.
5. **Lenguaje y funcionalidades gratuitos**.
6. **Soporte y actualización del *framework*.**

En este caso, debido a que toda la lógica del escaneo se encuentra en el *back-end* de la aplicación, para la creación de la aplicación web se han utilizado únicamente dos librerías:

1. **Prettier**: es una librería que permite darle formato a un texto plano, eligiendo el tipo de lenguaje, en este caso, sirve para darle formato al código HTML de los elementos en los que se han encontrado errores o vulnerabilidades.
2. **SASS.**

4.2. Identificación de requisitos

En este apartado, se expondrán y analizarán los **requisitos de la aplicación, funcionales y no funcionales**, para conocer los elementos que se han de tener en cuenta a la hora de realizar el desarrollo de la aplicación.

4.2.1. Requisitos funcionales

Tabla 1. Listado de requisitos funcionales de la aplicación.

Fuente: Elaboración propia.

Identificativo	Descripción
RF-01	La aplicación debe permitir a los usuarios introducir una URL, la cuál será escaneada y validada por la aplicación.
RF-02	La aplicación debe definir de forma clara los niveles de importancia de las alertas y errores detectados.
RF-03	La aplicación debe mostrar a los usuarios diferentes tipos de escaneos disponibles.
RF-04	La aplicación debe mostrar a los usuarios los informes creados hasta la fecha con un pequeño resumen de estos.
RF-05	La aplicación debe ser capaz de analizar las cabeceras HTTP presentes en la respuesta del servidor web.
RF-06	La aplicación debe detectar e identificar posibles configuraciones inseguras de las cabeceras HTTP.
RF-07	La aplicación debe de mostrar a los usuarios la información de las cabeceras de forma sencilla, con posibles valores correctos de estas.
RF-08	La aplicación debe mostrar a los usuarios información general sobre la página web, protocolo, dirección IP…

RF-09	La aplicación debe mostrar la información sobre el certificado SSL, como, por ejemplo, el emisor, la validez o la versión TLS.
RF-10	La aplicación debe incluir enlaces de interés que den más información y contexto a los usuarios.
RF-11	La aplicación debe ser capaz de generar más contexto y/o soluciones a los fallos encontrados mediante la integración con modelos de lenguaje.
RF-12	La aplicación debe ser capaz de analizar el código HTML en busca de patrones y fallos.
RF-13	La aplicación debe crear un tablero tipo Kanban con las vulnerabilidades o fallos encontrados durante el escaneo.
RF-14	La aplicación debe integrar filtros para las tareas dentro del tablero Kanban.
RF-15	Los usuarios podrán crear listas, eliminar y modificar las listas dentro del tablero Kanban.
RF-16	Los usuarios podrán crear, eliminar y mover las tareas dentro del tablero Kanban.
RF-17	La aplicación debe realizar consultas parametrizadas en la base de datos para evitar errores o posibles ataques de inyección.

4.2.2. Requisitos no funcionales

Tabla 2. Listado de requisitos no funcionales de la aplicación.

Fuente: Elaboración propia.

Identificativo	Descripción
RNF-01	La aplicación debe poder ejecutarse en cualquier sistema operativo, utilizando para este fin Docker como plataforma de despliegue.
RNF-02	La aplicación debe estar disponible de forma gratuita y ser de código abierto.
RNF-03	Los datos de la aplicación serán almacenados dentro de una base de datos MariaDB.
RNF-04	La aplicación debe contar con un API REST que utilice Python como lenguaje de programación.
RNF-05	La aplicación debe contar con una interfaz gráfica o página web desarrollada utilizando el framework de React, NextJS.
RNF-06	La aplicación web debe usar tecnologías que sean compatibles con los navegadores actuales.
RNF-07	El diseño de la interfaz debe ser amigable y entendible por todos los usuarios.
RNF-08	La aplicación debe ser navegable desde cualquier dispositivo, independientemente de las dimensiones de este.
RNF-09	Las comunicaciones o acciones que contengan datos críticos se deben realizar usando peticiones de tipo POST.

4.3. Diagrama de casos de uso

Para entender de una forma más sencilla y gráfica los requisitos funcionales de la aplicación, así como los distintos tipos de interacciones que ocurren entre el usuario y la aplicación, se ha creado un diagrama de casos de uso con las interacciones más importantes que se dan dentro de la aplicación.

Dentro del diagrama que se mostrará a continuación se pueden observar los siguientes elementos:

- **Actor**: es representado mediante el icono de una persona. Representa una persona o grupo de personas que cumplen un papel en la interacción con el sistema.
- **Caso de uso**: es representado mediante un círculo u óvalo con el nombre del caso de uso en su interior. Representa una funcionalidad o requisito del sistema
- **Relación**: es representado mediante una flecha que une dos casos de uso o un caso de uso y un actor. Representa las interacciones que se producen en el sistema.

En este caso, **solo hay un actor dentro de la aplicación**, el usuario, ya que, como es una aplicación que se **ejecuta de forma local**, no es necesario crear otros roles o usuarios diferentes.

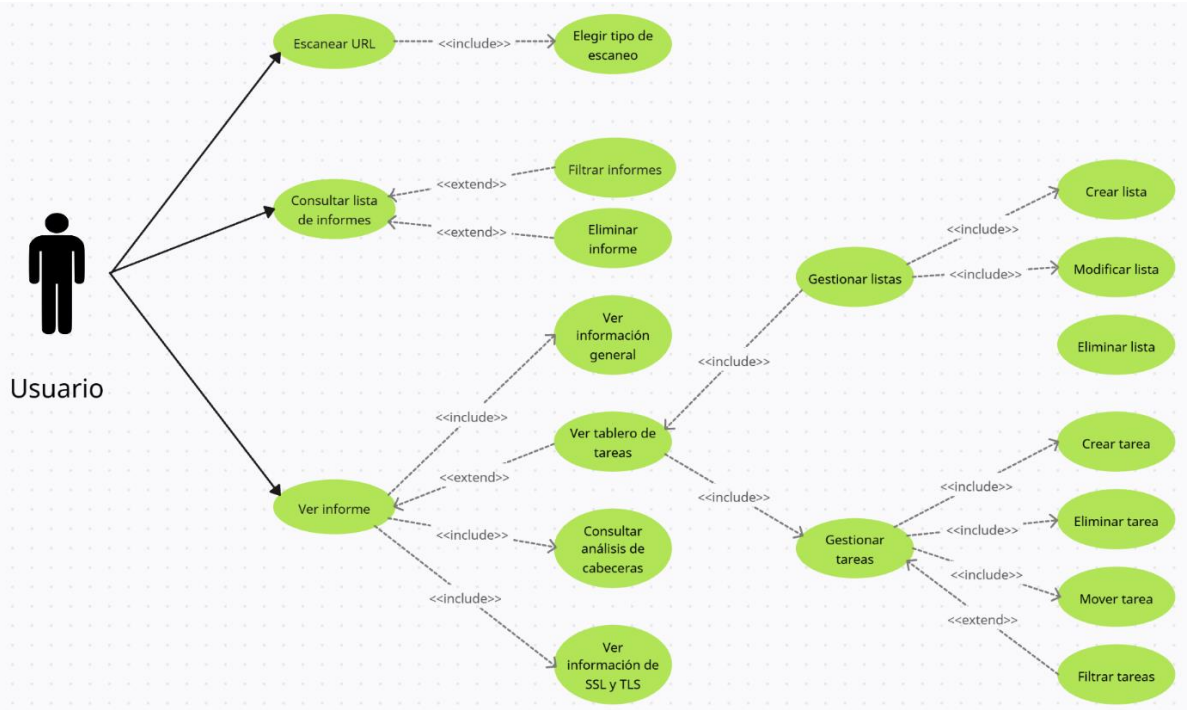


Figura 3. Diagrama de casos de uso de la aplicación.

Fuente: Elaboración propia.

Como se puede observar el usuario que accede a la aplicación, puede realizar cuatro tipos de interacciones principales:

1. **Escanear la URL**: cuando el usuario accede a la aplicación la vista principal de la aplicación permite al usuario introducir la URL que quiere escanear y además puede elegir el tipo de escaneo que quiere realizar.
2. **Consultar la lista de informes**: el usuario podrá acceder a una vista donde la aplicación muestra una lista con los informes que se han realizado hasta la fecha, dando la posibilidad de borrar de forma individual cualquiera de los informes listados o filtrarlos.
3. **Ver informe**: el usuario podrá acceder a la información detallada del informe generado por la herramienta, en la vista del informe podrá visualizar la información general del servidor web, información sobre el certificado SSL/TLS, información sobre las cabeceras y acceder al tablero para gestionar y visualizar las vulnerabilidades y fallos encontrados.
4. **Ver tablero**: el usuario podrá acceder al tablero Kanban generado por la aplicación, donde se encuentran todas las tareas y las listas, en esta vista el usuario podrá crear, modificar y eliminar listas además de mover, eliminar y crear tareas.

4.4. Desarrollo del API

La **API (Interfaz de Programación de Aplicaciones)** es la base de la herramienta desarrollada, debido a que actúa como “cerebro” de esta, esto se debe a que es la encargada de **unificar** y **crear un punto de acceso simple** a la lógica de negocio, integrando las distintas partes de la aplicación como son, la base de datos, el motor de escaneo y la interfaz de usuario.

Esta capa de la aplicación es la responsable de **recibir las peticiones del usuario**, llevar a cabo el proceso de **escaneo de vulnerabilidades y de almacenar** los resultados para su posterior consulta.

La estructura de archivos y carpetas que tiene la API es la que se muestra en la siguiente figura:

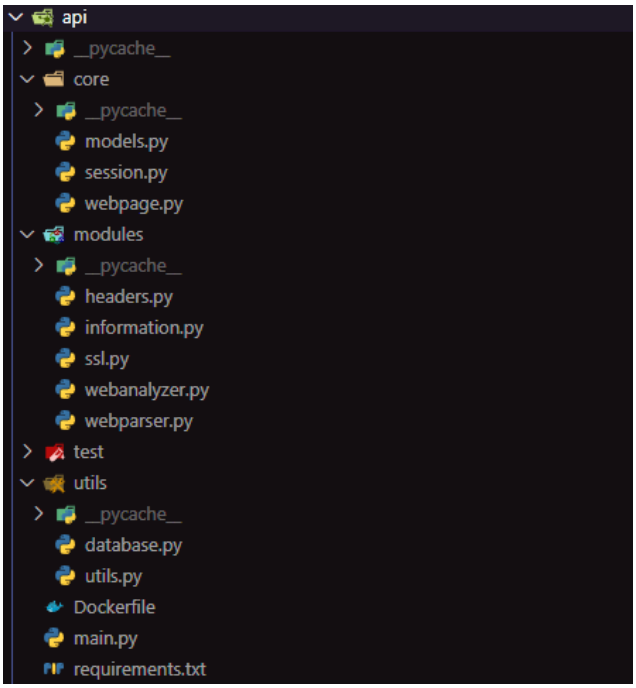


Figura 4. Estructura de los archivos y carpetas de la API de la aplicación.

Fuente: Elaboración propia.

En esta se puede observar el archivo "*main.py*", que cuenta con la lógica general de la API (métodos y rutas), la carpeta "*utils*", donde se encuentran diferentes funciones para operar en la base de datos y realizar acciones generales, como validar el dominio.

Por otro lado, está la carpeta "*modules*", donde residen todos los módulos que componen el motor de escaneo y por último la carpeta "*core*", con archivos para crear diferentes tipos de objetos, como el objeto que maneja la sesión o la información de la página web.

4.4.1. Funcionamiento del API

El funcionamiento general de la API, desarrollada utilizando la librería de Python FastAPI (4.1.1.1), se centra en un sistema de **rutas o "*endpoints*",** las cuales responden a peticiones HTTP que hace la aplicación y los usuarios de esta, manejando los posibles errores o códigos de error.

Cada una de las **rutas** de la API **está asociada a una función o funcionalidad específica** que procesa la solicitud e interactúa con los demás componentes, como el motor de escaneo, la base de datos y la interfaz de usuario, devolviendo finalmente una respuesta estructurada en

formato JSON. Esta estructura permite una c**omunicación eficiente entre la interfaz de usuario y lógica de negocio** de la aplicación.

En el desarrollo de este apartado se explicarán y mostrarán ejemplos de los diferentes métodos implementados en la API, así como ejemplos de llamadas y respuestas de esta.

4.4.1.1. Métodos implementados en la API

Gracias a la integración de la librería FastAPI, la implementación de los diferentes métodos ha sido **muy sencilla y ordenada** y ha permitido seguir el **estándar REST** (*Representational State Transfer*).

Los métodos se han asociado a las diferentes rutas de la aplicación según el fin para que se diseñaron, y son los siguientes:

- **GET**, es utilizado para **solicitar los datos de un recurso** mediante operaciones que implican **solo lectura**. En la figura mostrada a continuación, se recupera la información referente a una tarea específica, llamando a la ruta "GET /task/1", estando el identificador de la directamente en la ruta.

```python
@api.get("/task/{task_id}")
def get_task(task_id: str):
    db_connection = db_pool.get_connection()
    try:
        task = database.get_task_by_id(db_connection, task_id)
        if not task:
            raise HTTPException(
                status_code=404,
                detail={
                    "status": "error",
                    "message": f"Tarea {task_id} no encontrada."
                }
            )

        return {
            "status": "success",
            "message": f"Tarea {task_id} encontrada.",
            "data": {"task": task}
        }
    finally:
        db_connection.close()
```

Figura 5. Ejemplo de implementación de método GET con FastAPI.

Fuente: Elaboración propia.

- **POST**, este método es utilizado para **crear un nuevo recurso**, en este caso, se usa para insertar información dentro de la base de datos. En la figura mostrada a continuación, se crea una nueva tarea en base a la información que se adjunta en el cuerpo de la petición a la ruta "POST /task".

```python
@api.post("/task")
def create_task(task: TaskCreate):
    db_connection = db_pool.get_connection()
    try:
        task_id = database.create_task(db_connection, task)
        if not task_id:
            raise HTTPException(
                status_code=500,
                detail={
                    "status": "error",
                    "message": f"Error al crear la lista {task.title}"
                }
            )

        return {
            "status": "success",
            "message": f"Tarea '{task.title}' creada correctamente.",
            "data": {"task_id": task_id}
        }
    finally:
        db_connection.close()
```

Figura 6. Ejemplo de implementación de método POST con FastAPI.

Fuente: Elaboración propia.

- **PUT**, este método es utilizado para **actualizar un recurso existente**, en este caso, es utilizado para actualizar los datos de un informe, lista o tarea. En la figura mostrada a continuación, se actualizan los datos de una tarea en base a los campos e información que se adjuntan en el cuerpo de la petición a la ruta “PUT /task/1”, con esta función puede actualizarse de forma independiente el campo o atributo que se requiera.

```python
@api.put("/task/{task_id}")
def update_task(task_id: str, task: TaskUpdate):
    db_connection = db_pool.get_connection()
    try:
        update_fields = {key: value for key, value in task.dict().items() if value is not None}
        if not update_fields or len(update_fields) < 1:
            raise HTTPException(
                status_code=404,
                detail={
                    "status": "error",
                    "message": "No se proporcionaron campos válidos a actualizar."
                }
            )

        new_task = database.update_task(db_connection, task_id, update_fields)
        if not new_task:
            raise HTTPException(
                status_code=500,
                detail={
                    "status": "error",
                    "message": f"Error al actualizar la tarea {task_id} o el 'list_id' es incorrecto."
                }
            )

        return {
            "status": "success",
            "message": f"Tarea {task_id} actualizada correctamente.",
            "data": {"task": new_task}
        }
    finally:
        db_connection.close()
```

Figura 7. Ejemplo de implementación de método PUT con FastAPI.

Fuente: Elaboración propia.

- **DELETE**, este método se utiliza para **borrar o suprimir un recurso**, en la API se utiliza para borrar la información referente a informes, listas o tareas. En la figura mostrada a continuación, se elimina la información referente a una tarea en específico realizando una petición a la ruta “DELETE /task/1”.

```python
@api.delete("/task/{task_id}")
def delete_task(task_id: str):
    db_connection = db_pool.get_connection()
    try:
        deleted = database.delete_task(db_connection, task_id)
        if not deleted:
            raise HTTPException(
                status_code=500,
                detail={
                    "status": "error",
                    "message": f"Error al eliminar la tarea {task_id}."
                }
            )

        return {
            "status": "success",
            "message": f"Tarea {task_id} eliminada correctamente.",
            "data": {"task_id": task_id}
        }
    finally:
        db_connection.close()
```

Figura 8. Ejemplo de implementación de método DELETE con FastAPI.

Fuente: Elaboración propia.

4.4.1.2. Llamadas y respuesta de la API

Finalmente, tras haber explicado y enseñado algunos ejemplos de métodos y funciones que implementa la API, en este apartado se mostrarán **ejemplos concretos** de cómo se realiza la **interacción con la API** y las **respuestas que esta genera** y envía en formato JSON *(JavaScript Object Notation*).

Durante el desarrollo, para realizar pruebas sobre las rutas de la API de las cuales se concluía la implementación, se ha utilizado el navegador o directamente el software Postman.

Un ejemplo de la **llamada que crea una nueva tarea** es la siguiente, en la figura se muestra una captura sacada directamente de Postman para que se pueda observar el cuerpo del mensaje enviado y debajo la respuesta a la petición.

POST http://localhost:8000/task/



Figura 9. Ejemplo de cuerpo y respuesta para una llamada tipo POST usando Postman.

Fuente: Elaboración propia.

Un ejemplo de la llamada que **retorna la información completa sobre una tarea** en base a su identificador es la siguiente:

GET http://localhost:8000/task/598


```
{
    "status": "success",
    "message": "Tarea 598 encontrada.",
    "data": {
        "task": {
            "id": 598,
            "list_id": 10,
            "title": "Nueva Tarea Creada",
            "type": "Form",
            "severity": "Medium",
            "location": "location",
            "code": null,
            "details": "detalles",
            "status": 0,
            "archived": 0
        }
    }
}
```


Figura 10.Ejemplo de la respuesta para una llamada tipo GET usando Postman.

Fuente: Elaboración propia.

4.4.2. Base de datos

La base de datos está construida utilizando **MariaDB** y es la forma que tiene la aplicación de **lograr persistencia de los datos**, en esta, se guarda la información de sobre los reportes, las listas del tablero de gestión y las tareas que se encuentran en las listas.

A la hora de realizar las consultas a la base de datos se usan lo que se denominan **consultas parametrizadas,** las cuales separan el código SQL de los datos que se van a insertar en la consulta, ofreciendo **seguridad frente a diferentes ataques** o errores.

En la siguiente figura se puede observar un ejemplo de consulta, en donde se elimina uno de los informes generados por la herramienta utilizando su identificador.

```python
def delete_report(db_connection, report_id):
    try:
        cursor = db_connection.cursor()

        sql = "DELETE FROM Report WHERE id = ?"
        data = (report_id,)
        cursor.execute(sql, data)
        db_connection.commit()
        deleted = cursor.rowcount > 0

        cursor.close()
        return deleted
    except Exception as e:
        return False
```

Figura 11. Método para la eliminación de un informe.

Fuente: Elaboración propia.

Para evitar errores y que la **base de datos** pueda manejar **más de una operación de forma simultánea**, se ha creado un "*pool*" de hilos, que consiste en una estructura que almacena hilos de trabajo, con una instancia de conexión a la base de datos, los cuales están listos para ejecutar tareas.

```python
def create_db_pool(threads: int) -> ConnectionPool:
    load_dotenv()

    for connection_try in range(10):
        try:
            pool = ConnectionPool(
                pool_name="mypool",
                pool_size=threads,
                user=os.environ['DB_USER'],
                password=os.environ['DB_PASSWORD'],
                host=os.environ['DB_HOST'],
                port=int(os.environ['DB_PORT']),
                database=os.environ['DB_NAME']
            )

            print(f"INFO: Connection pool created, size: {threads}")
            print("INFO: Connected to MariaDB!")
            return pool
        except mariadb.Error as e:
            print(f"INFO: Try {connection_try+1}/10, {e}")
            time.sleep(3)
```

Figura 12. Método para la creación de un pool de hilos con conexión a la base de datos.

Fuente: Elaboración propia.

Durante la ejecución de la aplicación, en lugar de crear un hilo nuevo por cada solicitud que realiza el cliente, la aplicación gestiona y reparte un hilo por cada conexión que se realiza con la API, le asigna una tarea y tras finalizar, devuelve el hilo al "*pool*".

Este proceso se puede observar en la Figura 12, donde primero se asigna un hilo de la conexión y tras terminar la operación se cierra y devuelve al "*pool*".

Las **razones** por las que se ha optado por esta solución son las siguientes:

1. **Mejora del rendimiento de la aplicación**, esto se debe a que el "pool" está ya creado al iniciar la aplicación, por lo que las consultas pueden procesarse de forma instantánea gracias a que no se necesita crear y destruir la conexión entre una llamada y otra.
2. **Aumento de la concurrencia**, se pueden procesar múltiples llamadas de forma simultánea.
3. **Gestión de recursos controlada**, debido a que se puede elegir número de hilos que tendrá la aplicación.

4.5. Desarrollo del motor de escaneo y análisis

El **motor de escaneo y análisis** es el elemento que proporciona a la aplicación el servicio principal que implementa, es decir, la **búsqueda y análisis de fallos y vulnerabilidades**. Este motor utiliza algunas de las librerías citadas en apartados anteriores, para dar la funcionalidad buscada a la aplicación.

En los siguientes apartados, se realizará un desglose de todas las funcionalidades con las que la herramienta cuenta.

4.5.1. Análisis de cabeceras

Una de las acciones iniciales que realiza la aplicación a la hora de empezar con el escaneo de la página web, es la comprobación de que el dominio o URL dados son accesibles y están funcionando de forma correcta, una vez realizada esta comprobación será realizado el análisis de cabeceras.

4.5.1.1. Inicio de conexión y objeto de sesión

La aplicación cuenta con un **objeto de Python** que tiene como finalidad **gestionar de una forma eficiente las peticiones que se realizan a una URL o dominio**. Las funcionalidades básicas de este objeto son las siguientes:

1. Formatear la URL o dominio introducidos por el usuario para que esté en un formato que la aplicación pueda manejar.
2. Comprobar la existencia y funcionamiento del servidor web al que se va a realizar la petición.
3. Guardar información sobre la petición HTTP, con el fin de ser reutilizada en otros módulos y funciones de la aplicación.

Para comprobar que el dominio introducido por el usuario resuelve, es decir, es accesible y funciona de forma correcta, la aplicación crea un objeto de sesión vacío y primero, verifica que el **dominio introducido es válido**, usando el método "***set_domain***", tras esto utiliza el método "***make_request***" para comprobar que el **dominio existe y guardar toda la información** necesaria.

```
session = Session()
db_connection = db_pool.get_connection()

try:
    if session.set_domain(domain):
        if session.make_request():
```

Figura 13. Fragmento de código para la inicialización del objeto "session".

Fuente: Elaboración propia.

El método "*make_request*" utiliza la librería mencionada anteriormente, *Requests* (4.1.1.2), la cual permite realizar la petición HTTP al dominio y guardar la información necesaria en el objeto de sesión.

```python
def make_request(self):
    error = None
    schemas = ["https", "http"]

    for schema in schemas:
        print(f"INFO: Intentando acceder a {schema}://{self.domain}")
        full_domain = f"{schema}://{self.domain}"
        try:
            response = requests.get(full_domain, timeout=10)
            status = response.status_code

            if status == 200:
                self.schema = schema
                self.status = status
                self.full_domain = full_domain
                self.response = response

                print(f"INFO: Dominio {full_domain} accedido - status: {status}")
                break
            else:
                print(f"ERROR: Fallo al acceder a {full_domain} - status: {status}")
        except RequestException as e:
            print(f"ERROR: El dominio {full_domain} no resuelve o no es accesible")
            error = str(e)

    if not self.response:
        if error:
            self.error = error
        self.valid = False
        return False
```

Figura 14. Método para la ejecución de la petición HTTP sobre el dominio requerido.

Fuente: Elaboración propia.

Es importante recalcar que este método también tiene en cuenta que el dominio introducido no tenga un certificado SSL válido disponible y es por esto por lo que, en caso de que el dominio no sea accesible mediante HTTPS, realiza otra prueba mediante HTTP.

Tras haber completado la ejecución del método, la **información que se guarda dentro del objeto** de sesión es la siguiente:

1. Dominio.
2. URL introducida por el usuario.
3. Esquema utilizado, puede ser HTTP o HTTPS.
4. Puerto, si existe.
5. Respuesta a la petición realizada
6. Estado de la petición HTTP.
7. Error, si ha salido algo mal.
8. Validez del dominio introducido.

4.5.1.2. Tipos de cabeceras y su análisis

Una vez establecido el objeto de sesión, dentro del atributo "*response*", se guarda toda la información referente a la respuesta de la petición HTTP, siendo muy importante en este caso las cabeceras de la petición.

Una mala configuración de las cabeceras puede dar pie a la explotación de diferentes vulnerabilidades o errores en el servidor o aplicación web.

La herramienta hace un **análisis de las diez cabeceras** que se han considerado más importantes y críticas en el ámbito de seguridad utilizando el **módulo** denominado "***headers***". Este análisis consiste en la comprobación de que las cabeceras están o no presentes y si, además, tienen un valor seguro o correcto configurado.

Las cabeceras que analiza la herramienta son las siguientes:

1. **HTTP Strict-Transport-Security (HSTS**), es una característica de seguridad que permite a un sitio web indicar a los navegadores que sólo se debe comunicar con HTTPS en lugar de usar HTTP (MDN, Strict-Transport-Security, 2025).
2. **Content-Security-Policy (CSP**), es una capa de seguridad adicional que ayuda a prevenir y mitigar algunos tipos de ataque, incluyendo Cross Site Scripting (XSS) y ataques de inyección de datos (MDN, Content Security Policy (CSP), 2025)
3. **X-Frame-Options**, para indicar si se permite a un navegador renderizar una página en un objeto o etiqueta embebido, as páginas web pueden usarlo para evitar ataques de *click-jacking* (MDN, X-Frame-Options, 2025).
4. **Content-Type-Options**, es un marcador utilizado por el servidor para indicar que los los tipos MIME definidos en la cabecera Content-Type no se deben cambiar, evitando así que se ejecuten archivos como si fueran código, cuando no deberían (MDN, X-Content-Type-Options, 2025).
5. **Set-Cookie**, es una cabecera usada para enviar cookies desde el servidor al usuario y se usa para definir configuraciones se seguridad en ese intercambio, evitando que las cookies sean accesibles desde HTTP y que no lo sean desde Javascript (MDN, Set-Cookie, 2025).
6. **Cache-Control**, es una cabecera que contiene o define directivas o instrucciones, para controlar el almacenamiento temporal (MDN, Cache-Control, 2025).

7. **X-XSS-Protection**, es una cabecera que ofrece una protección adicional contra ataques de tipo Cross-Site Scripting (XSS), en algunos navegadores (MDN, X-XSS-Protection, 2025).
8. **Referrer-Policy**, define que tipo de datos se comparten entre las diferentes solicitudes realizadas y limitar la información compartida (MDN, Referrer-Policy, 2025).
9. **Permissions-Policy**, proporciona un mecanismo para permitir o denegar el uso de funciones del navegador en un documento, como el acceso a la cámara (MDN, Permissions-Policy header, 2025).
10. **Refresh**, esta cabecera indica al navegador que actualice o redirija la página, una vez transcurrido un tiempo especificado, aunque no es peligrosa en sí puede ser utilizada de forma maliciosa (MDN, Refresh header, 2025).

A continuación, se muestra una de las diez funciones que sirven para analizar y comprobar que el valor de la cabecera es correcto. Para ello, se han investigado los valores de seguridad para cada una de las cabeceras y se ha comprobado que los valores configurados por el usuario coincidan con estos.

```python
def check_referrer(headers: dict) -> dict:
    referrer = headers.get("referrer-policy")
    result = {
        "severity": "recommendation",
        "name": "Referrer-Policy",
        "description": "Indica qué información se envía a otros sitios cuando el usuario hace clic en un enlace. Sin esta cabe
        "recommendation": "no-referrer, strict-origin-when-cross-origin, no-referrer-when-downgrade, strict-origin",
        "more": "https://developer.mozilla.org/es/docs/Web/HTTP/Headers/Referrer-Policy"
    }

    if referrer:
        isCorrect = False

        if referrer not in ['no-referrer', 'strict-origin-when-cross-origin', 'no-referrer-when-downgrade', 'strict-origin']:
            isCorrect = True

        result["value"] = referrer
        result["enabled"] = True
        result["correct"] = isCorrect
    else:
        result["enabled"] = False
        result["correct"] = False

    return result
```

Figura 15. Método para el análisis de la cabecera "Referrer".

Fuente: Elaboración propia.

Para la cabecera Referrer-Policy los valores que se comparan y son recomendados son "*no-referrer*", "*strict-origin-when-cross-origin*", "*no-referrer-when-downgrade*", "*strict-origin*" (Nalpas, 2020).

Para cada una de estas cabeceras la aplicación devuelve si la **cabecera está activada**, si es **correcto**, su **nombre**, **descripción**, los **posibles valores recomendados** y un **enlace** que da más información y contexto al usuario.

Además, la aplicación cuenta con un **nivel de severidad** para cada una de las **alertas** o vulnerabilidades reportadas, estos niveles son los siguientes:

1. Informativo o recomendado
2. Bajo
3. Medio
4. Crítico

Estos niveles ayudan a los usuarios a priorizar y **entender de un forma gráfica y rápida** que **avisos son los más importantes o críticos**.

4.5.2. Información general y certificado SSL/TLS

Para aportar más profundidad y datos al informe generado por la herramienta, se añade información adicional, que, aunque no está directamente relacionada con la seguridad de la página web y el servidor, puede ser de utilidad.

4.5.2.1. Información general

Esta sección del informe recopila información sobre el servidor o página web que **ayuda a contextualizar** todos los demás **datos que se han recopilado del escaneo**. Esta información es recopilada por el módulo de la aplicación que se llama "*information*".

Dentro de la información recopilada y mostrada se encuentra:

1. Fecha del escaneo.
2. Dominio y URL.
3. Dirección IP y alias del servidor.
4. Tecnología del servidor, si está presente.
5. Tipo de generador de contenido (CMS), si existe.

```python
def get_information(domain, port, headers):
    socket = get_socket(domain, port)
    server_info = get_server(headers)
    ip_info = list(set(socket[2])) if socket[2] else None
    alias_info = socket[1] if socket[1] else None

    return {
        "ip": ip_info,
        "alias": alias_info,
        "server": server_info["server"],
        "powered": server_info["powered_by"],
        "generator": server_info["generator"]
    }
```

Figura 16. Método para la obtención de información general de un servidor web.

Fuente: Elaboración propia

4.5.2.2. Certificado SSL

Además de la información general, la aplicación recopila y muestra la información referente al **certificado SSL/TLS de la página**, utilizando el módulo que se llama "*ssl*".

Tal y como se muestra en la siguiente figura, la aplicación utiliza diferentes librerías como pyOpenSSL (4.1.1.5), para conseguir y dar formato a la información recopilada que posteriormente será mostrada al usuario. Esta información es de gran utilidad ya que puede avisar al usuario de que el certificado está cerca de expirar.

```python
def analyze_ssl(domain: str, schema: str) -> dict:
    if schema == "http":
        return {"error": "not_supported", "message": "SSL/TLS no es compatible con HTTP"}

    domain = utils.remove_path(domain)
    context = ssl.create_default_context()
    certificate = get_certificate(context, domain)
    tls = get_tls_info(context, domain)
    x509 = OpenSSL.crypto.load_certificate(OpenSSL.crypto.FILETYPE_PEM, certificate)

    result = {
        'subject': dict(x509.get_subject().get_components()),
        'issuer': dict(x509.get_issuer().get_components()),
        'serial_number': x509.get_serial_number(),
        'version': x509.get_version(),
        'not_before': x509.get_notBefore().decode('utf-8'),
        'not_after': x509.get_notAfter().decode('utf-8'),
        'tls_version': tls['version'],
        'tls_cipher': tls['cipher'],
    }
```

Figura 17. Método para el análisis de la información SSL.

Fuente: Elaboración propia.

La información mostrada en la aplicación es la siguiente:

1. Sujeto y emisor del certificado.
2. País.
3. Fechas de validez.
4. Versión y algoritmo de cifrado TLS.

4.5.3. Análisis sintáctico del código HTML

Por último, una de las funcionalidades más importantes del motor de escaneo es o son los módulos encargados de recopilar y dar formato al código HTML de la página con el fin de analizarlo en busca de patrones, vulnerabilidades o fallos de configuración. Los módulos encargados de este fin son "*webparser*" y "*webanalyzer*".

4.5.3.1. Extracción de información y objeto de página web

El primer paso a la hora de analizar el código estático de una aplicación es conseguir el código que se quiere analizar y extraer la información necesaria de este código.

En la herramienta se crea un **objeto en Python que almacena y gestionar toda la información necesaria sobre el código HTML** de una página, con la finalidad de tener una **estructura homogénea** a la hora de analizar el contenido sin importar el formato o atributos que tenga la página que se quiere escanear.

Este objeto llamado "***WebPage***", a su vez, está **compuesto de otros objetos**, tal y como sucede en la vida real, como, por ejemplo, formularios de la página web, a su vez compuesto de objetos de tipo campo, enlaces, etiquetas a recursos externos… teniendo estos objetos sus propios atributos, métodos e información asociada.

Este objeto tiene, además, funciones útiles como "***load_webpage***" la cual, mediante la integración de la librería Playwright (4.1.1.3), consigue acceder a la página web y cargar el contenido estático desde el lado del cliente, para su posterior análisis.

```python
async def load_webpage(self, url: str):
    async with async_playwright() as p:
        browser = await p.chromium.launch()
        page = await browser.new_page()
        await page.goto(url)
        await page.wait_for_load_state("load")

        self.content = await page.content()

        await browser.close()
```

Figura 18. Método para la carga del contenido de la página web requerida.

Fuente: Elaboración propia.

Una vez almacenado el contenido de la página dentro del objeto, se usa el módulo "***webparser***" para **extraer, formatear y guardar toda la información**. Este módulo utiliza la librería BeautifulSoup (4.1.1.4), que incorpora numerosas funcionalidades para hacer la tarea de extracción mucho más fácil.

Como se puede observar en la figura a continuación, se utiliza el módulo "*html.parser*" de la librería para crear el objeto que contiene toda la información necesaria y, a continuación, se llama a funciones encargadas de extraer y acceder a la información de cada uno de los elementos.

```python
def parse_webpage(webpage: WebPage):
    if not webpage.content:
        return {}

    soup = BeautifulSoup(webpage.content, 'html.parser')

    parse_links(soup, webpage)
    parse_metatags(soup, webpage)
    parse_scripttags(soup, webpage)
    parse_linktags(soup, webpage)
    parse_forms(soup, webpage)
```

Figura 19. Método para la extracción de la información del contenido de la página web.

Fuente: Elaboración propia.

Estas funciones se encargan de recorrer el contenido en busca de la o las etiquetas deseadas, extrayendo y guardando la información, haciendo las operaciones y comprobaciones oportunas, para finalmente guardar el resultado en el objeto "WebPage".

Un ejemplo se muestra en la siguiente figura, en la función "*parse_links*", se iteran todos y cada uno de los enlaces de la página web (etiqueta <a>) y se extrae información, como la página a la que apunta el enlace, el texto y demás atributos de la etiqueta.

```python
def parse_links(soup: BeautifulSoup, webpage: WebPage):
    print("INFO: Parsing links in the webpage")
    for link in soup.find_all('a'):
        link_href = link.get('href')
        link_text = link.get_text().strip()
        link_target = link.get('target')
        link_rel = link.get('rel')

        #print('[LINK]', link_href, link_text, link_rel, link_target)
        webpage.add_link(Link(link_href, link_text, link_rel, link_target, str(link)))
```

Figura 20. Método para la extracción de la información de los enlaces de una página.

Fuente: Elaboración propia.

4.5.3.2. Análisis de información y detección de vulnerabilidades

Una vez extraído la información y con el formato correcto, la aplicación utiliza el módulo "***webanalyzer***" para **analizar todas estas etiquetas y elementos HTML** en busca de posibles errores o vulnerabilidades.

Cada una de las vulnerabilidades detectadas son **almacenadas en el objeto** "**Vulnerability**" para posteriormente insertarlas en la base de datos y mostrarlas al usuario.

Este objeto posee información como el nombre y tipo de la vulnerabilidad, por ejemplo, una vulnerabilidad puede ser de tipo enlace o formulario, la severidad de esta, que sigue una escala de niveles como la vista en el apartado 4.5.1.2, la localización y código de la vulnerabilidad y un texto que da más información y detalles acerca de esta.

```python
class Vulnerability:
    def __init__(self, name: str, type: str, severity: str, details: str, location: str = "", code: str = ""):
        self.name = name
        self.type = type
        self.severity = severity
        self.location = location if location is not None else ""
        self.code = code if code is not None else ""
        self.details = details
```

Figura 21. Código del objeto "Vulnerability".

Fuente: Elaboración propia.

El módulo de análisis de la página web, tiene diferentes funciones para **analizar cada una de las etiquetas y elementos deseados**. Cada una de estas funciones itera estos elementos en busca de posibles vulnerabilidades y guarda todas las encontradas.

En la figura a continuación, se muestra el código de la función "***analyze_links***", la cual se encarga de detectar vulnerabilidades por la presencia de links externos que apuntan a un sitio inseguro y enlaces externos que no tienen atributos de seguridad configurados.

```python
def analyze_links(webpage: WebPage):
    print("INFO: Analyzing links in the webpage")

    for link in webpage.links:
        # Enlace externo inseguro
        if link.href and link.href != '' and link.href.startswith('http://'):
            webpage.add_vulnerability(Vulnerability(
                name="Enlace externo inseguro (HTTP)",
                type="Enlace",
                severity="Low",
                location=link.href,
                code=link.code,
                details="El enlace apunta a un recurso externo utilizando HTTP en lugar de HTTPS, lo que puede indicar que el recurso pertene
            ))

        # Link con target _blank sin rel="noopener noreferrer"
        rel_value = ""
        if link.blank and link.href is not None and link.href != "" and link.external:
            if link.rel is None or link.rel is not None and "noopener" not in link.rel and "noreferrer" not in link.rel: rel_value = "rel=noo
            elif "noopener" not in link.rel: rel_value = "rel=noopener"
            elif "noreferrer" not in link.rel: rel_value = "rel=noreferrer"

            if rel_value != "":
                webpage.add_vulnerability(Vulnerability(
                    name=f"Enlace con target _blank sin {rel_value}",
                    type="Enlace",
                    severity="Medium",
                    location=link.href,
                    code=link.code,
                    details="El enlace tiene un target '_blank' sin el atributo 'rel' recomendado, Esto puede permitir que la página abierta
                ))
```

Figura 22. Método para el análisis de los enlaces de una página web.

Fuente: Elaboración propia.

La aplicación es capaz de detectar y reportar hasta veintidós **vulnerabilidades, errores de configuración o malas prácticas en el código** HTML de una página. Es importante recalcar que la herramienta no realiza ningún ataque activo a esta, y todas las vulnerabilidades encontradas son con una única petición a la página web.

La **herramienta puede detectar los siguientes casos**:

1. Ausencia de token anti-CSRF (Cross-Site Request Forgery) en un formulario.
2. Atributo "*action*" del formulario inseguro, es decir, apunta a un recurso externo potencialmente inseguro que usa HTTP.
3. Envío de datos sensibles utilizando el método GET en un formulario.
4. Atributo "*action*" del formulario vacío.
5. Campo de contraseña en un formulario como texto plano.
6. Envío de contraseñas utilizando el método GET en un formulario, para el caso anterior.
7. Carga de recursos (etiqueta <script>) desde URLs inseguras que usan HTTP.
8. Ausencia de la configuración de seguridad Subresource Integrity (SRI) en scripts.
9. Carga de recursos (etiqueta <link>) desde URLs inseguras que usan HTTP.
10. Ausencia de configuración de seguridad Subresource Integrity (SRI) en links.
11. Uso de atributos obsoletos en links.

12. Falta de Política de Seguridad de Contenidos (CSP)
13. Redirección de la meta etiqueta o cabecera Refresh a un recurso externo inseguro que usa HTTP.
14. Ausencia o mala configuración de atributo de la meta etiqueta "*charset*".
15. Ausencia de la meta etiqueta "*robots*".
16. Ausencia o configuración insegura de la política de Referrer.
17. Exposición de información con meta etiqueta "*generator*".
18. Enlaces a sitios externos inseguros que usan HTTP.
19. Enlaces con el atributo "*target*" presente con valor "*_blank*" pero que no posee atributos de seguridad.

En la figura que se muestra a continuación se puede observar el orden de ejecución de cada una de las funciones y módulos expuestos en los anteriores apartados.

```
webpage = WebPage(session.domain)
await webpage.load_webpage(session.full_domain)
webparser.parse_webpage(webpage)
webanalyzer.analyze_webpage(webpage, result_headers)
```

Figura 23. Fragmento de código con los métodos para el análisis y extracción de información sobre la página web.

Fuente: Elaboración propia.

4.6. Desarrollo de la interfaz de usuario

La interfaz de usuario es la capa de presentación de la aplicación, siendo esta el p**unto de interacción** directo que existe ente los **usuarios y la herramienta de escaneo**. A través de una interfaz clara, sencilla y simple, los usuarios interesados pueden acceder y ejecutar las funcionalidades de la herramienta, así como visualizar la información que esta genera.

Es por esto, que los principios de diseños se han enfocado en la usabilidad y accesibilidad, asegurando que la información técnica generada por el motor de análisis sea presentada de forma comprensible.

4.6.1. Estructura de la aplicación

Next.js facilita la creación de interfaces de usuario robustas y eficientes gracias a su enrutamiento basado en archivos y su capacidad para renderizar en el servidor (SSR) o generar sitios estáticos (SSG), garantizando así que la aplicación sea rápida.

La estructura de archivos y carpetas que tiene la interfaz de usuario es la que se muestra en la siguiente figura:

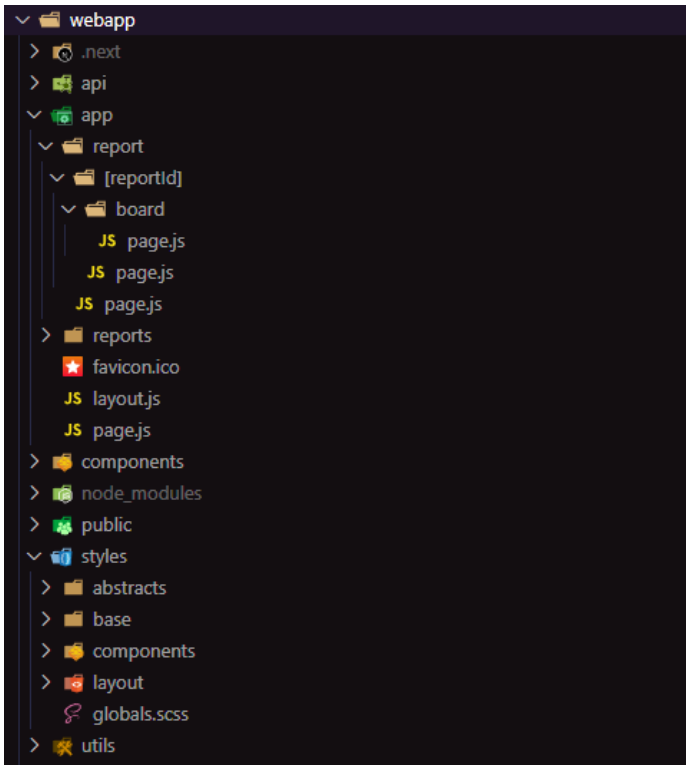


Figura 24. Estructura de los archivos y carpetas de la interfaz de la aplicación.

Fuente: Elaboración propia.

La estructura de carpetas de Next.js, con la carpeta "app/", organiza de una forma lógica e indexada las interfaces de la aplicación, siendo cada uno de los archivos **"page.js", el archivo principal que renderiza una página.**

Además, en este proyecto se cuenta con otras carpetas como son "components", "styles", "public", "utils" o "api" con las llamadas necesarias a la API desarrollada.

4.6.1.1. SASS, temas y diseño modular

Para gestionar los estilos de la página, se ha integrado SASS (Syntactically Awesome Style Sheets) junto con Next.js. SASS extiende la funcionalidad base de CSS y permite crear variables, selectores anidados, funciones, importaciones y "*mixins*", lo que **ayuda a conseguir modularidad** que se busca con el diseño de la web y permite crear temas de colores para la interfaz.

4.6.1.2. Componentes y reutilización

El enfoque de componentes de React, la base de Next.js, es fundamental para desarrollar una interfaz de usuario modular y bien estructurada.

Esto permite encapsular las funciones, eventos, lógica e interfaz de una parte del sistema, ayudando al que **sistema sea mucho más eficiente y reutilizable** a la hora de incorporar nuevas funcionalidades.

```
export default function InputDomain({onSubmitScan}) {
    const [domain, setDomain] = useState('')

    const handleSubmit = (event) => {
        event.preventDefault()
        event.target.elements['domain'].blur()
        if (onSubmitScan) onSubmitScan(domain)
    }

    const handleChange = (event) => { setDomain(event.target.value) }

    return (<>
        <p className="input-label" htmlFor="domain">Introduce la URL de la web</p>
        <form className="input-domain" onSubmit={handleSubmit}>
            <input
                id="domain"
                onChange={handleChange}
                className="input-text"
                type="text"
                placeholder="https://example.com o example.com"
            />
            <button
                className="input-submit button button-primary"
                type="submit"
            >
                Escanear
            </button>
        </form>
```

Figura 25. Código del componente "InputDomain" de la aplicación.

Fuente: Elaboración propia.

4.6.2. Vistas de la aplicación

Como se indicado anteriormente, con Next.js se pueden definir vistas o páginas directamente desde la propia estructura de archivos de la aplicación. En este proyecto era necesario diseñar y desarrollar varias vistas para crear una experiencia de usuario fluida y completa.

Todas las páginas de la aplicación comparten la misma disposición o *layout*, tienen una barra de navegación, el contenido de la página y un pie de página.

4.6.2.1. Página principal (Home)

La **página principal** de la aplicación es el punto de entrada para los usuarios, desde la cual se podrán **realizar los escaneos**. El diseño de esta página tiene como finalidad presentar de forma sencilla los elementos necesarios para realizar un escaneo, en este caso, la URL de la página web y el tipo de escaneo, estos elementos se encuentran agrupados dentro de una tarjeta.

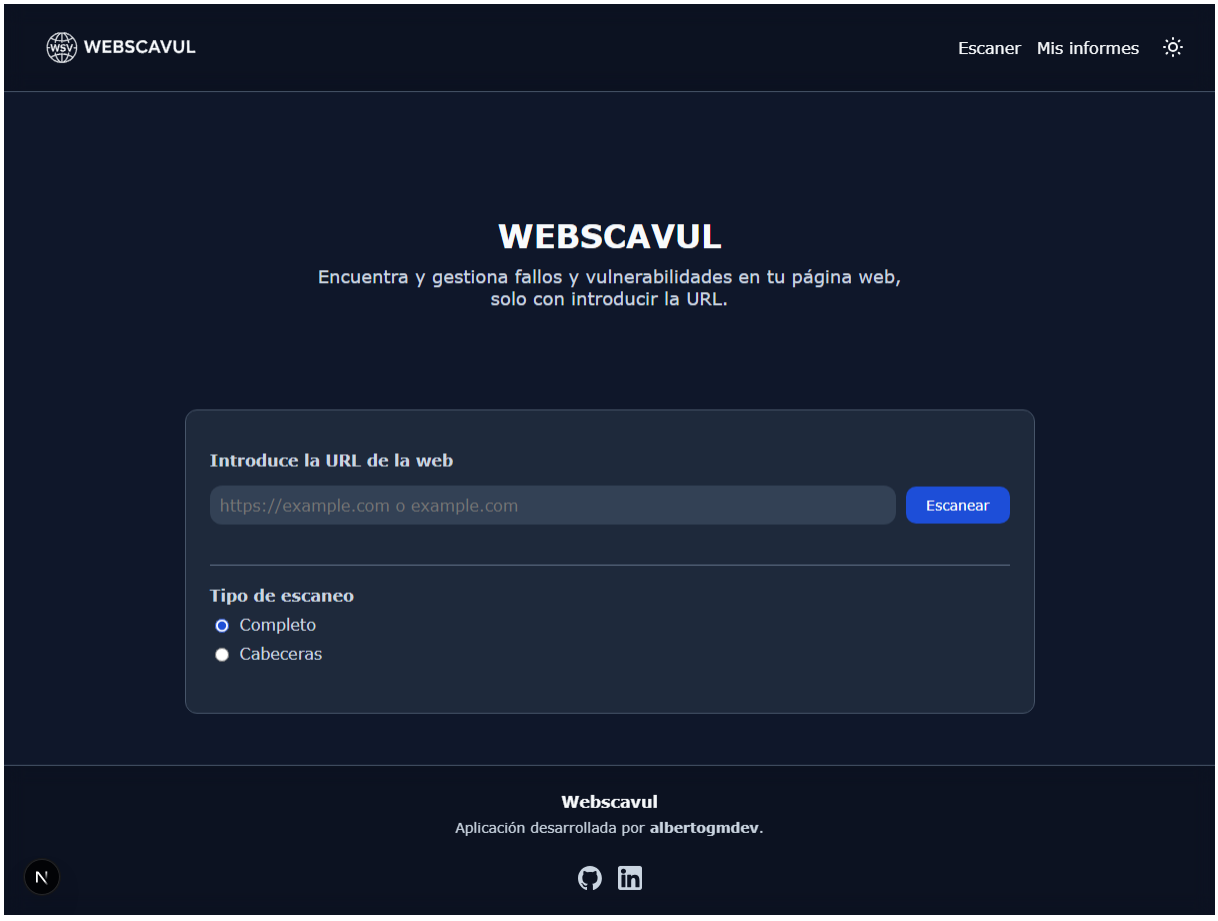


Figura 26. Página principal (Home) de Webscavul.

Fuente: Elaboración propia.

Una vez que el usuario ha introducido la URL y elegido el tipo de escaneo, la aplicación, antes de realizar el escaneo, comprueba que la URL o dominio introducido sea válido, en caso de que no lo sea, avisa el usuario con un mensaje de advertencia.

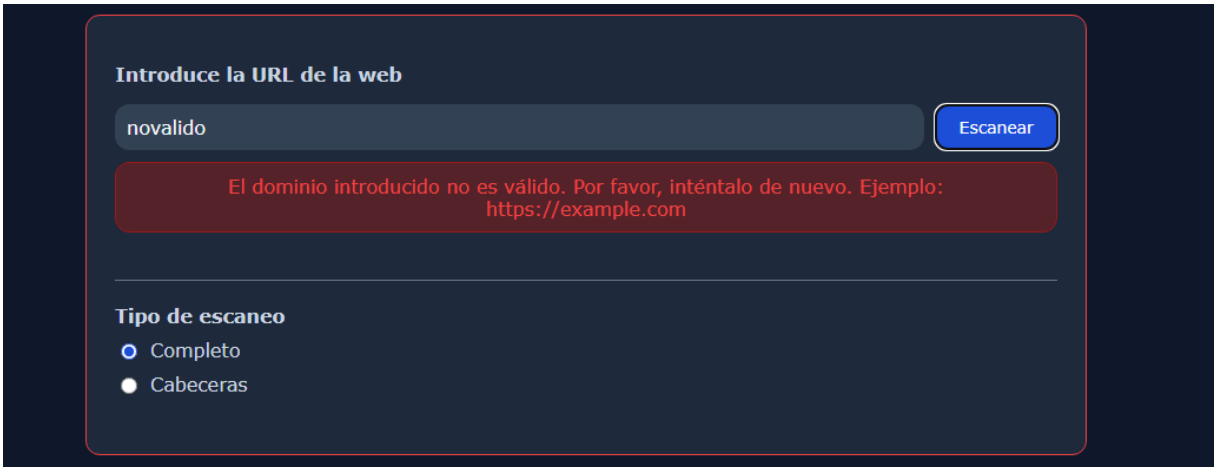


Figura 27. Error mostrado por la aplicación tras introducir un dominio no válido.

Fuente: Elaboración propia.

Si el dominio introducido es válido se inicia el proceso de escaneo, para ello, la aplicación hace una petición a la API de la aplicación indicando la URL introducida y el tipo de escaneo a realizar.

Mientras el motor de escaneo está realizando la tarea, se le da retroalimentación al usuario mostrando un *"overlay"* sobre la tarjeta, con el fin de que este sepa que su solicitud está siendo procesada correctamente.

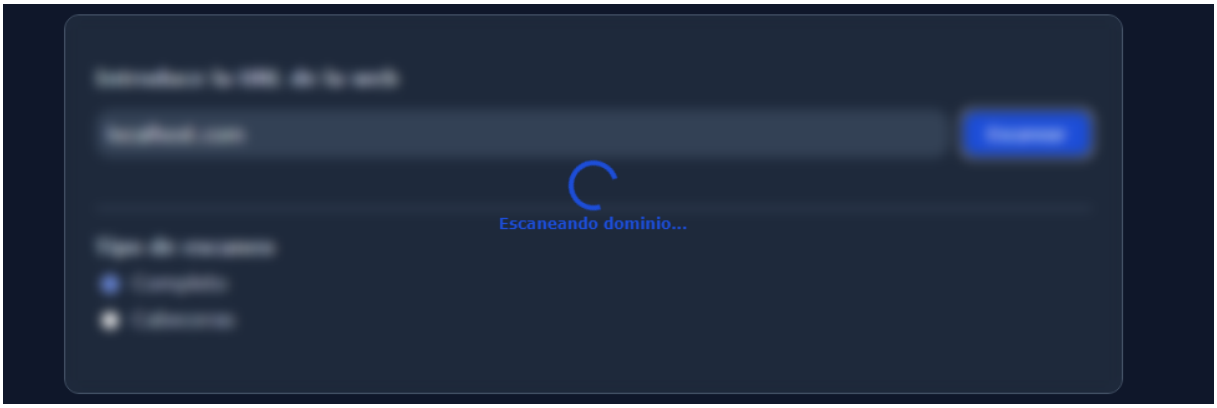


Figura 28. Mensaje mostrado mientras la aplicación realiza un escaneo.

Fuente: Elaboración propia.

Una vez termina el proceso de escaneo, si este se ha completado correctamente, la entrada en la base de datos con el informe habrá sido creada y la aplicación redirigirá directamente al usuario a la vista de detalle del informe (4.6.2.3), para que este puede visualizar de forma inmediata los resultados.

En caso de que haya habido algún error, durante la ejecución del escaneo y no haya sido posible crear el informe, un mensaje de error será mostrado al usuario para avisarle de que algo ha salido mal durante el proceso.

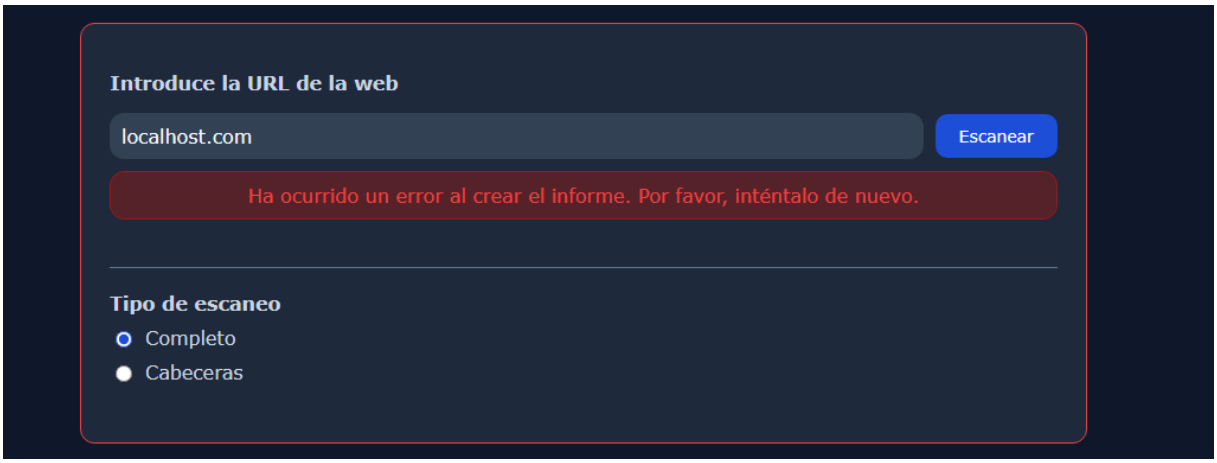


Figura 29. Error mostrado por la aplicación tras ocurrir un error en el escaneo.

Fuente: Elaboración propia.

4.6.2.2. Listado de informes

Cuando el o los usuarios de la aplicación han realizado uno o varios escaneos, se muestra un **listado de los escaneos en el apartado de la página web "Mis informes".** Desde esta vista, el usuario puede gestionar y visualizar de forma rápida los escaneos que ha realizado y acceder a la información específica del informe o al tablero creado por la aplicación.

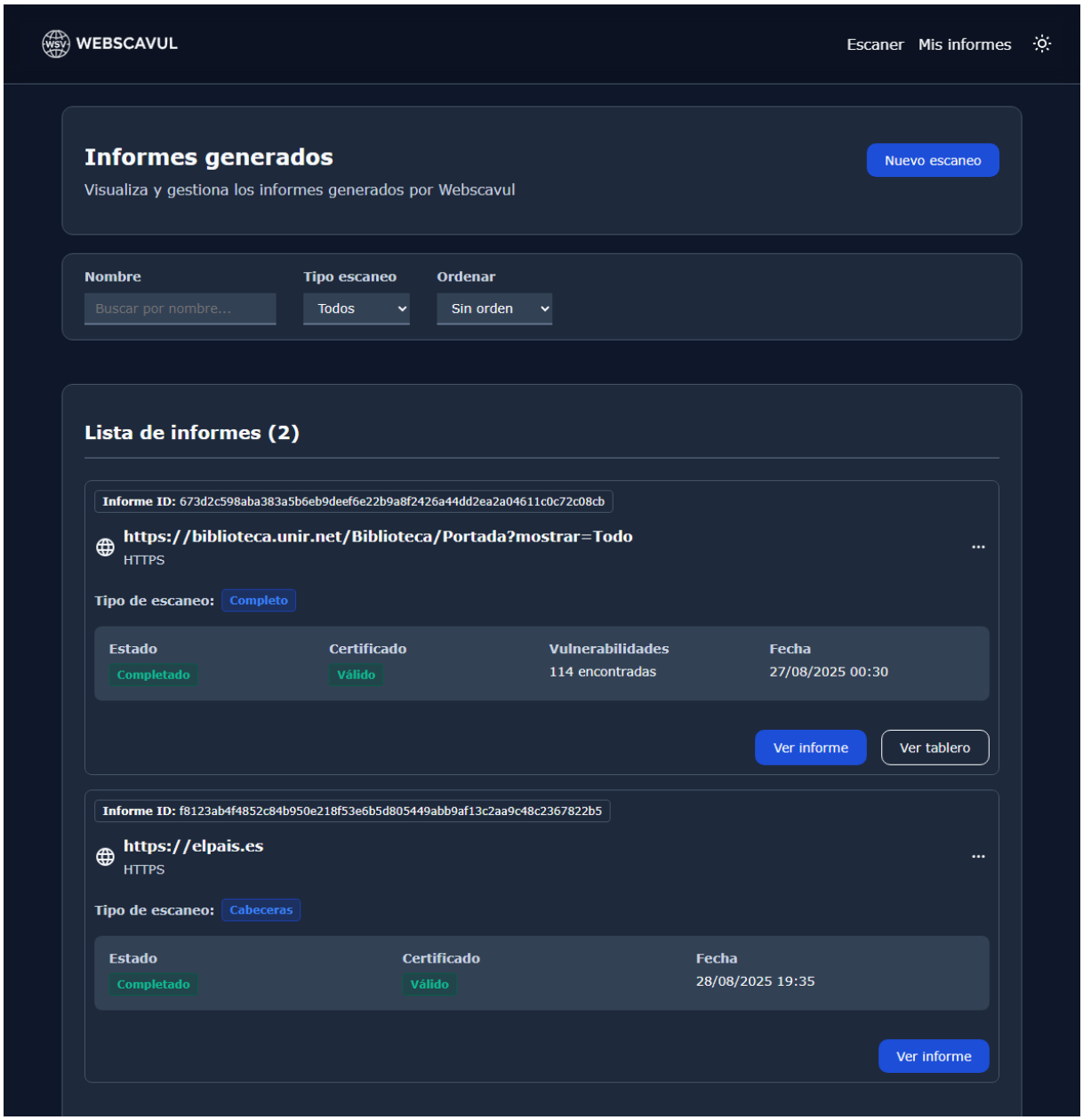


Figura 30. Página con el listado de informes de Webscavul.

Fuente: Elaboración propia.

Para cada uno de los escaneos realizados la aplicación muestra la **información más relevante**, **facilitando así la identificación rápida** del informe que se desea consultar.

Además, se incorpora un apartado de filtrado que permite refinar la búsqueda mediante criterios como el nombre del escaneo, el tipo de análisis o la fecha de realización, ofreciendo así una navegación más ágil y eficiente.

Para cada uno de los informes realizados, se muestra la siguiente información:

1. Identificador del informe
2. URL y protocolo
3. Tipo de escaneo
4. Estado del escaneo, si está completo o en curso.
5. Estado del certificado SSL, si es válido o no.
6. Números de vulnerabilidades encontradas.
7. Fecha de realización del escaneo.

Cada una de las entradas del listado, tiene un botón para ver el tablero de vulnerabilidades, en caso de que sea un escaneo completo, un botón para ir al detalle del informe y un menú de acciones que permite eliminar el informe si el usuario lo desea, tal y como se muestra en la siguiente figura.

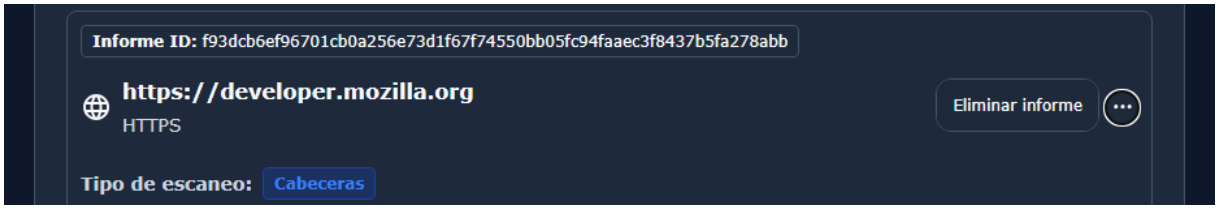


Figura 31. Menú desplegable con opción para eliminar el informe.

Fuente: Elaboración propia.

4.6.2.3. Detalle del informe

Tras realizar un escaneo o cuando el usuario requiere **consultar la información general y del análisis de las cabeceras,** se accede a la **vista con el detalle del informe**.

En esta página se muestra de manera estructurada la información recopilada y procesada durante el análisis, permitiendo que el usuario pueda revisar de forma clara, rápida y sencilla los resultados.

Al inicio de la página, se presenta un resumen general del informe y del dominio sobre el que se ha realizado el escaneo (dirección IP, alias, protocolo…), así como el número de vulnerabilidades encontradas, el número total de cabeceras analizadas, el estado del certificado SSL de la página y una sección con información ampliada sobre el certificado SSL y TLS del servidor.

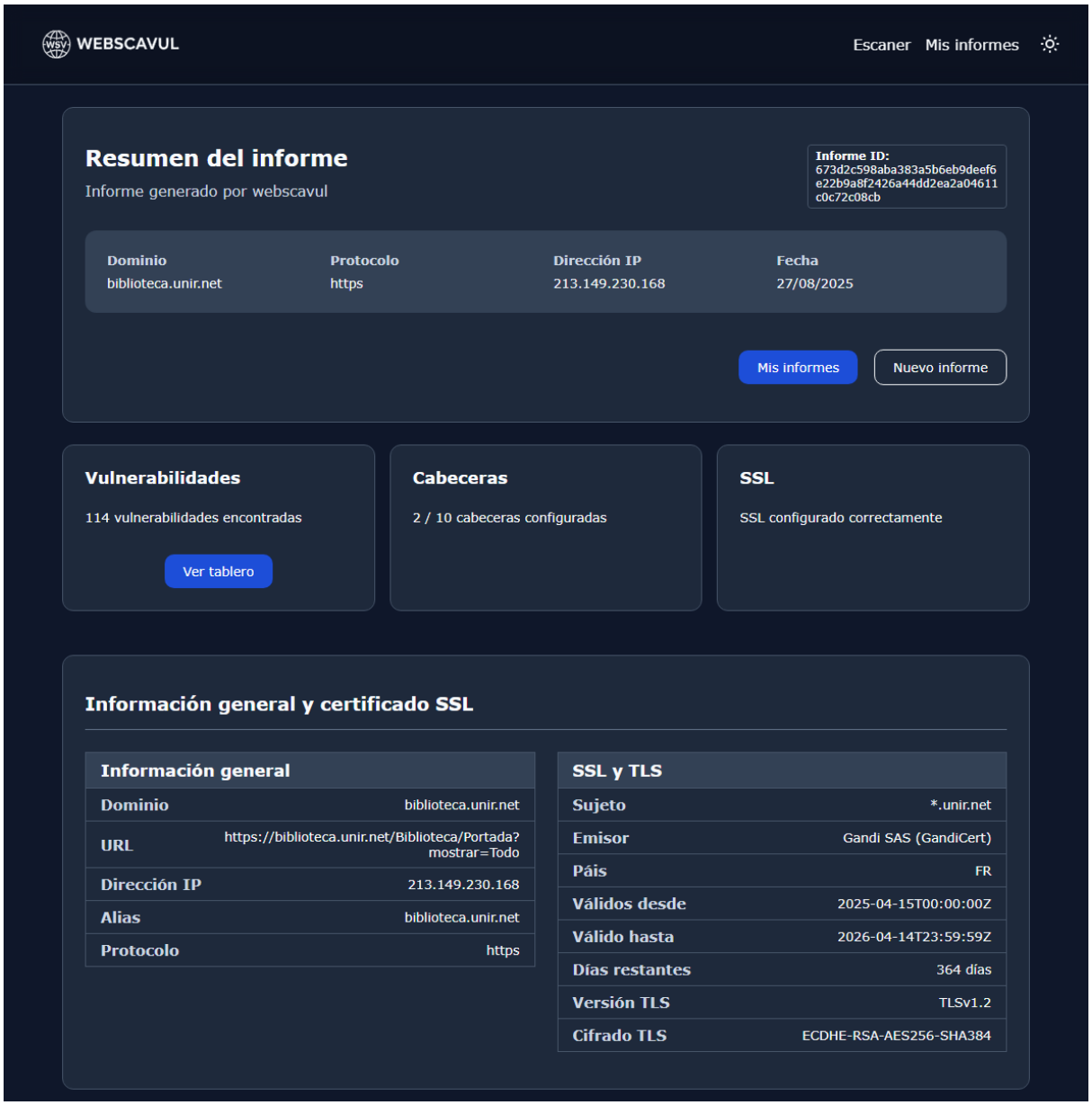


Figura 32. Página con el detalle del informe de Webscavul.

Fuente: Elaboración propia.

Tras esto, se encuentra el listado de las cabeceras analizadas durante la ejecución del escaneo, para cada una de estas cabeceras se muestra la siguiente información:

1. Nombre de la cabecera.
2. Descripción.
3. Valor actual de esta y el valor recomendado por la aplicación.
4. Nivel de severidad o indicador de valor correcto de la cabecera.
5. Enlace con más información y contexto sobre la cabecera.

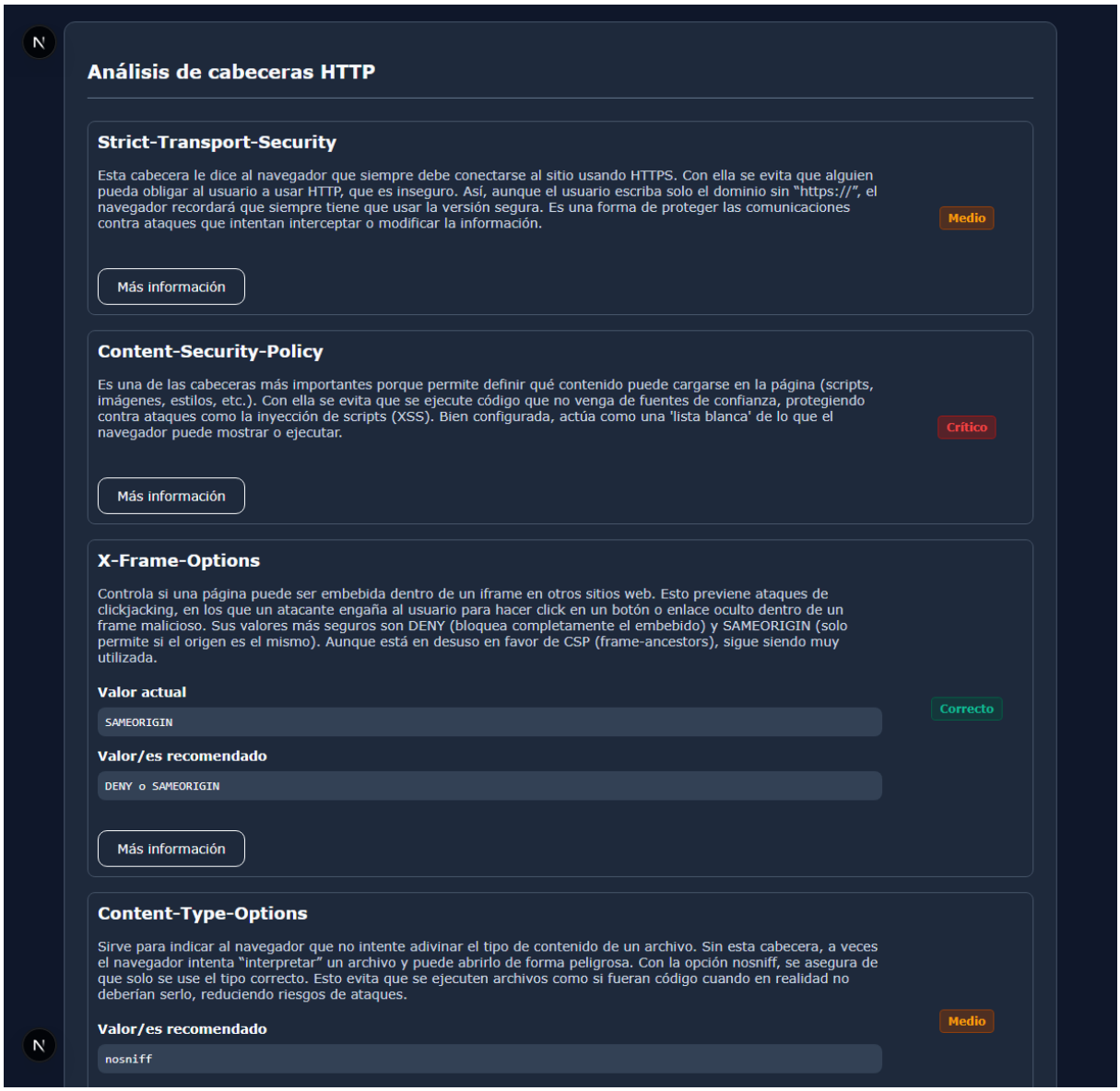


Figura 33. Listado con el resultado del análisis de las cabeceras realizado por Webscavul.

Fuente: Elaboración propia.

4.6.2.4. Tablero del informe

La aplicación genera un **tablero Kanban para facilitar a los usuarios la gestión y corrección de las vulnerabilidades y errores detectados**. Cada uno de los escaneos realizados por la aplicación tiene su propio tablero independiente, siempre y cuando se haya elegido del tipo de escaneo completo.

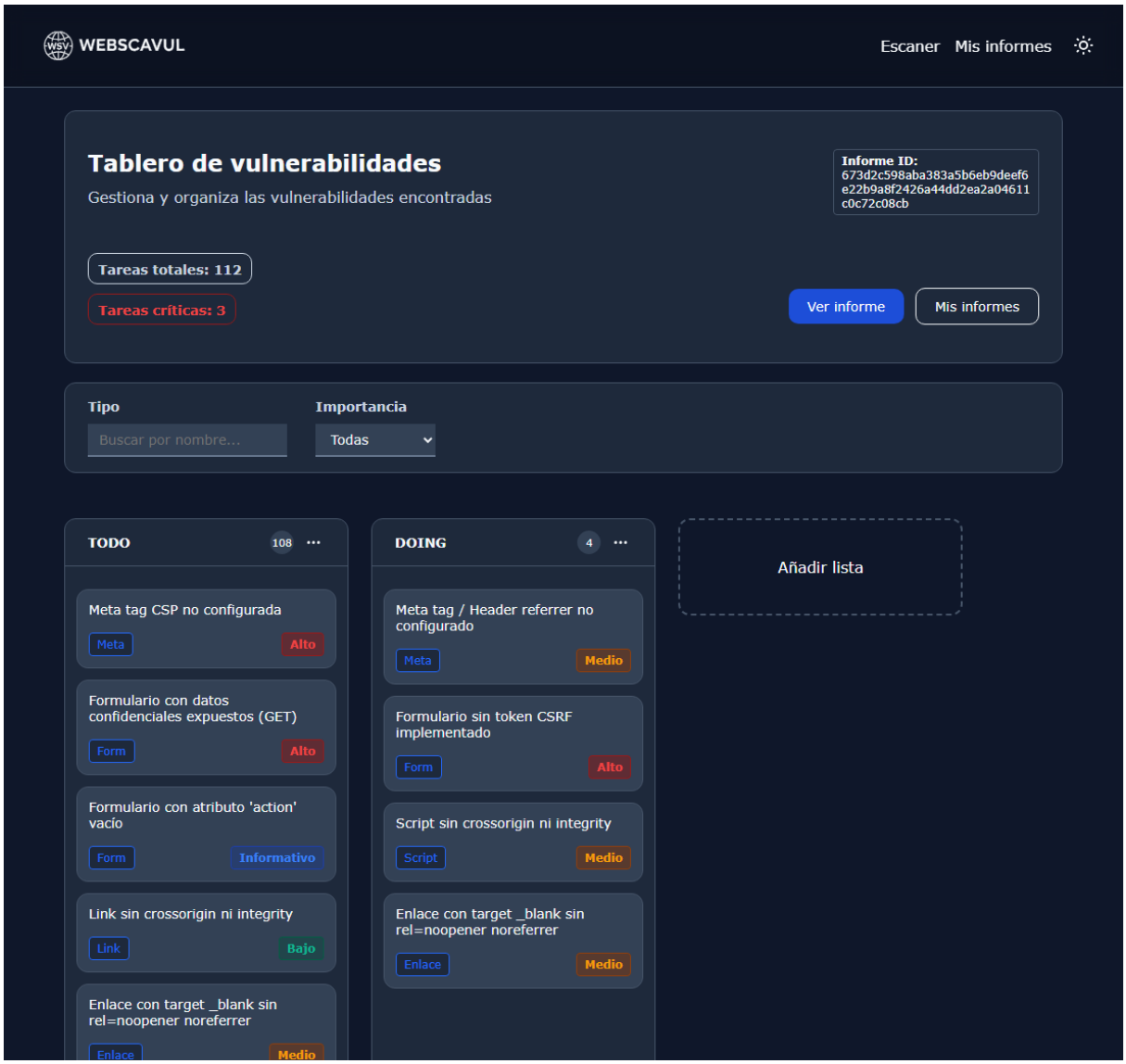


Figura 34. Página con el tablero de vulnerabilidades de Webscavul.

Fuente: Elaboración propia.

El tablero está compuesto por listas, pudiendo el usuario añadir una nueva lista al tablero, pulsando en el botón “Añadir lista”, tras esto, se abrirá un modal con un campo para introducir el nombre de la nueva lista.



Figura 35. Modal para la creación de una nueva lista.

Fuente: Elaboración propia.

Cada una de las listas del tablero tiene la información sobre el nombre o título de la lista, el número de tareas dentro de esta, las tareas que la componen y un botón que abre el menú de acciones de la lista.

Las acciones posibles que el usuario puede realizar son eliminar la lista actual, junto con todas sus tareas o editar el nombre de la lista. Si el usuario elige la opción "Editar lista" se abrirá un modal con la misma estructura que el mostrado en la Figura 35.

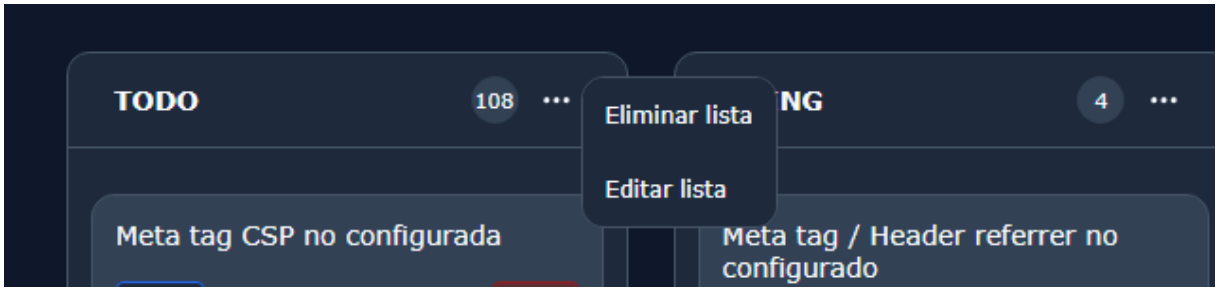


Figura 36. Menú desplegable con la opción de eliminar y editar una lista.

Fuente: Elaboración propia.

Las **tareas** dentro del tablero representan cada una de las **vulnerabilidades o fallos encontrados** por la aplicación, para cada una de estas, se muestra una información resumida; el nombre de la vulnerabilidad, el tipo o a que etiqueta del código afecta y la importancia o severidad de esta.

Si el usuario pulsa en una tarea, se abrirá un modal con información detallada de esta, incluyendo lo siguiente:

1. Tipo e importancia.
2. Detalles o explicación de la vulnerabilidad.
3. Ubicación de esta, identificador, ruta…
4. Código de la etiqueta afectada.
5. Opciones para generar más información o soluciones usando modelos de lenguaje ("¿Qué es esto?" o "¿Cómo solucionarlo?").

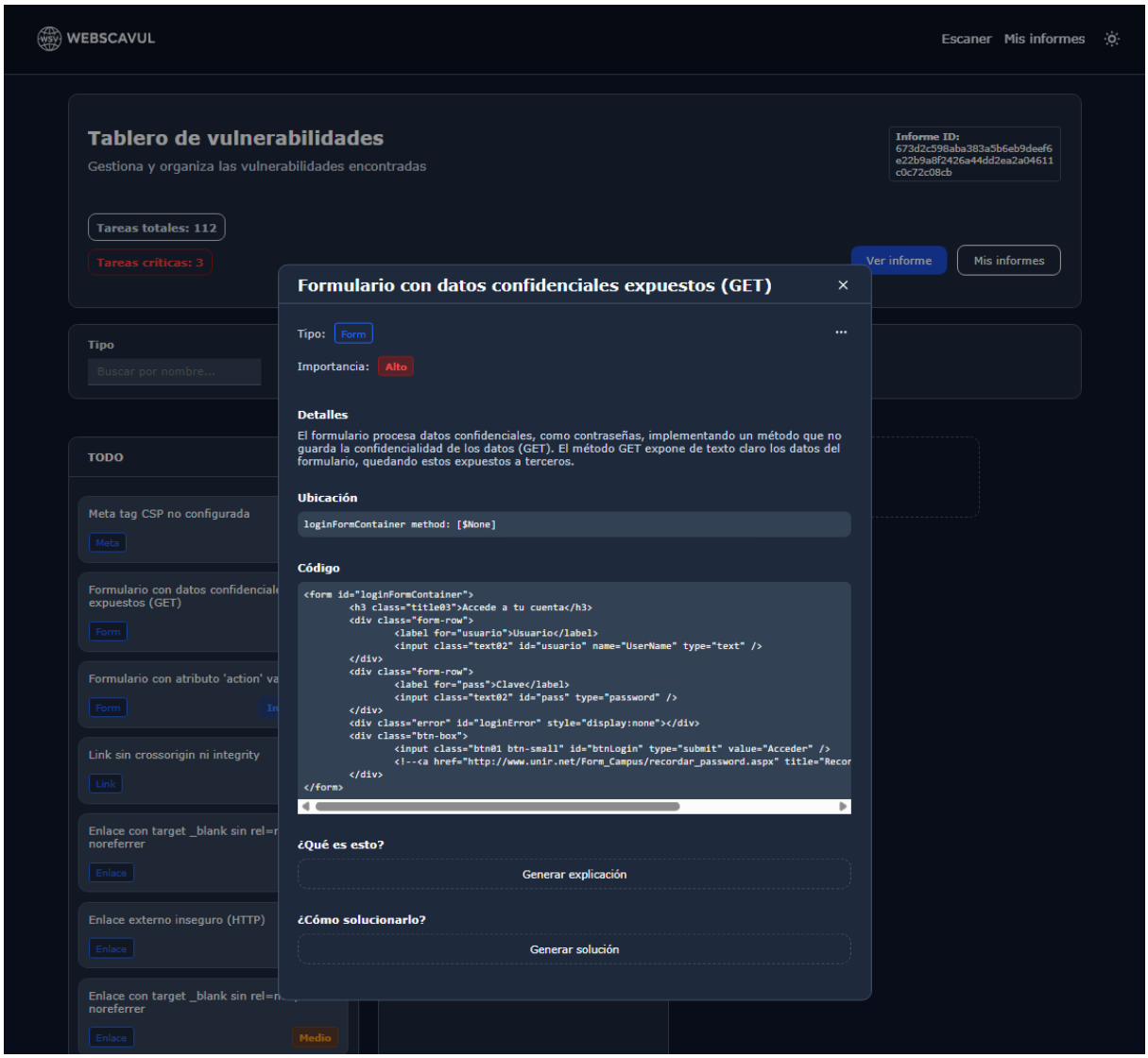


Figura 37. Modal con la información detallada de una vulnerabilidad.

Fuente: Elaboración propia.

El modal de la tarea tiene un botón para abrir el menú de acciones, que da al usuario a la posibilidad de eliminar una tarea o moverla a cualquiera de las listas creadas.

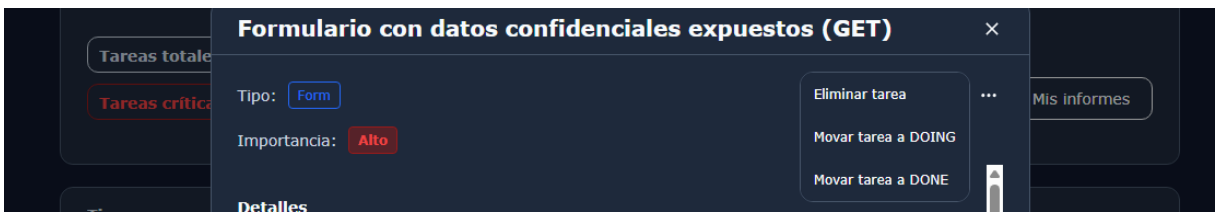


Figura 38. Menún desplegable con la opción de eliminar y mover una tarea.

Fuente: Elaboración propia.

Por último, dentro del modal, se le da la opción al usuario de **generar**, usando modelos de lenguaje, **soluciones o más contexto sobre la tarea o vulnerabilidad** que está visualizando, pulsando en el botón “Generar explicación” o “Generar solución”.

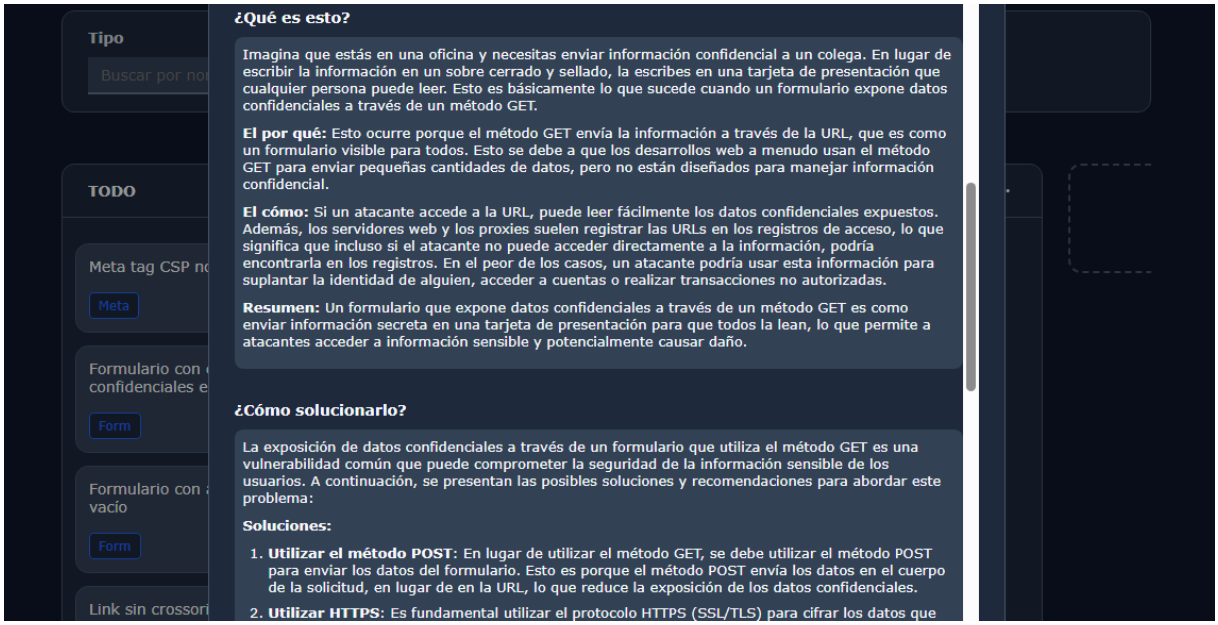


Figura 39. Ejemplo de solución y contexto generado por Webscavul usando modelos de lenguaje. Fuente: Elaboración propia.

El proceso de generación de esta información se explica de forma más detallada en el apartado sobre la generación de contexto y soluciones con modelos de lenguaje (4.6.4).

4.6.3. Integración con la API

La **integración con la API** constituye un aspecto fundamental de la aplicación, ya que permite **establecer la comunicación entre el *front-end* y el *back-end***. No se entrará en detalles sobre los métodos o funciones debido a que han sido explicados previamente (4.4).

Todas las llamadas a la API se manejan desde “*/api/index.js*”, en este archivo se encuentra un **método general (*fetchData*) que ayuda a crear la petición** según los parámetros indicados, tal y como se muestra en la siguiente figura, donde se implementa la función que elimina un informe.

```
export const deleteReport = async (reportId) => {
    try {
        const endpoint = `/report/${reportId}`
        const response = await fetchData(endpoint, {method: 'DELETE'})
        return response
    } catch (error) {
        console.error(`Failed to fetch from ${endpoint}:`, error);
        throw error;
    }
}
```

Figura 40. Método para la ejecución de la llamada a la API que elimina un informe.

Fuente: Elaboración propia.

Una vez creada la función que maneja la petición, se importa en el componente o vista en el que se necesite realizar dicha acción y se maneja la respuesta, ejecutando las acciones necesarias posteriormente.

```
const handleReportDelete = async (reportId) => {
    try {
        const response = await deleteReport(reportId)

        if (response.status === 200) {
            const afterDelete = reportData.filter(report => report.id !== reportId)

            setReportData(afterDelete)
            setFilteredReportData(afterDelete)
            // When modifying the board data, we also need to apply the current filters
            applyFilters(filterName, filterType, filterOrder, afterDelete)
            return true
        }
        else {
            console.error("Error deleting report")
            return false
        }
    } catch (err) {
        console.error("Error deleting report")
        return false
    }
}
```

Figura 41. Método que maneja la información recibida tras la eliminación de un informe.

Fuente: Elaboración propia.

4.6.4. Generación de contexto y soluciones con modelos de lenguaje

Otra de las funcionalidades principales de la aplicación, es la **generación de soluciones y/o contexto sobre las vulnerabilidades encontradas** con el fin de **facilitar la comprensión de los resultados obtenidos**. Esta característica busca ofrecer explicaciones claras y sencillas y recomendaciones prácticas que permitan al usuario interpretar correctamente el alcance de cada vulnerabilidad.

Con esto, se pretende **romper la barrera técnica y la dificultad de análisis** que de forma habitual tienen algunas herramientas de escaneo, siendo de ayuda tanto a perfiles especializados y más técnico, como a usuario con menos conocimientos en el ámbito de ciberseguridad o desarrollo web.

Para realizar la integración con modelos de lenguaje, se ha usado Groq, una plataforma, con una capa gratuita, que permite la integración de modelos de inteligencia artificial en aplicaciones y que cuenta con un conjunto de herramienta y librerías nativas (SDK) compatible con Next.js.

Toda la lógica que forma la consulta y el contexto de esta se encuentra en la ruta "*/utils/groq.js*", desde este archivo, se ejecutan las siguientes acciones:

1. Iniciación de la conexión usando la clave de la API generado en la plataforma.
2. Generación de un contexto en función de lo que el usuario solicite, que servirá al modelo para generar una respuesta más útil.
3. Ejecución de la consulta y devolución del resultado.

```
const groq = new Groq({
    apiKey: process.env.GROQ_API_KEY,
})

export async function askModel(query, type) {
    let systemContext = ""
    if (type === "solution") systemContext = `Eres un experto en ciberseguridad y desarrollo web, especializado en la identificación y mitigación de vuln
        Cuando se te proporcione una vulnerabilidad web, debes:
        Ofrecer las posibles soluciones en forma de pasos o configuraciones técnicas.
        Proporcionar ejemplos de código relevantes (HTML, JavaScript, PHP, Python, etc.) que ilustren las soluciones si es absolutamente necesario, en ca
        Recomendar las mejores prácticas y consideraciones adicionales para una implementación segura.

        A continuación, se te proporcionará la vulnerabilidad, con su nombre, importancia, información ya proporcionada y posible fragmento de código aso

    else if (type === "context") systemContext = `Eres un experto en ciberseguridad y un educador paciente, con la misión de explicar conceptos técnicos
        Cuando se te proporcione una vulnerabilidad que el usuario no entiende, debes:
        Utilizar analogías o ejemplos de la vida cotidiana para relacionar el concepto con algo familiar.
        Explicar el "por qué" de la vulnerabilidad: qué la causa y qué error de diseño o configuración la permite.
        Describir el "cómo": cómo un atacante podría explotarla y qué tipo de daño podría causar en caso de que sea una vulnerabilidad explotable por un
        Resumir la idea principal en una oración simple, como si estuvieras explicando el concepto a un amigo.

        A continuación, se se te proporcionará la vulnerabilidad, con su nombre, importancia, información ya proporcionada y posible fragmento de código

    try {
        const response = await groq.chat.completions.create({
            model: "llama-3.3-70b-versatile",
            messages: [
                { role: "system", content: systemContext },
                { role: "user", content: query }
            ],
        })

        return response.choices[0].message.content;
    } catch (error) {
        console.error("Error consultando AI:", error)
        throw error
```

Figura 42. Fragmento de código para la creación y ejecución de las consultas al modelo de lenguaje. Fuente: Elaboración propia.

Por último, será necesario implementar el método anterior cuando el usuario solicite a la aplicación generar una solución o contexto. En la siguiente figura, se muestra cómo se usa la función "*askModel*" y como se forma la consulta.

```
const handleAskModel = async (type) => {
    try {
        if (type === "solution" && taskSolution) return
        if (type === "context" && taskContext) return

        const query = `Vulnerabiliad: ${task.title}, Importancia: ${task.severity}, Información proporcionada: ${task.details}, Código asociado: ${task.code || "No hay código asociado."}`
        const modelAnswer = await askModel(query, type)

        if (type === "solution") seTaskSolution(modelAnswer)
        else if (type === "context") seTaskContext(modelAnswer)
    } catch (error) {
        console.error("Error fetching from AI:", error)
    }
}
```

Figura 43. Método que maneja la respuesta recibida del modelo de lenguaje.

Fuente: Elaboración propia.

4.7. Despliegue de la aplicación

Para evitar las posibles incompatibilidades y errores que pueden surgir durante el proceso de despliegue de la aplicación se ha optado por utilizar la **tecnología de contenedores Docker**.

Docker es una plataforma que permite **empaquetar una aplicación junto con todas las librerías, dependencias, tecnologías o configuraciones** que la componen en un entorno ligeramente aislado llamado contenedor, que, gracias al aislamiento, permite que varios contenedores se ejecuten de forma simultánea en el un mismo host.

Permite la virtualización ligera de las aplicaciones debido a que, a diferencia de la virtualización tradicional, la cual utiliza mucho espacio y recursos de la máquina en la que se está ejecutando, los contenedores no requieren un sistema operativo completo por cada instancia, sino que comparten el núcleo del sistema anfitrión, lo que reduce el consumo de recursos y mejora la eficiencia.

Las principales ventajas del uso de Docker son:

1. **Portabilidad** de la aplicación desarrollada, permite la ejecución de la aplicación en cualquier sistema operativo, compatible con Docker y garantiza que no existan diferencias entre los entornos de desarrollo y producción.
2. **Aislamiento**, cada contendor puede ejecutarse de manera independiente evitando así conflictos entre aplicaciones y versiones de librerías o dependencias.

3. **Rapidez de despliegue**, gracias al uso de imágenes se puede levantar una aplicación de forma rápida y sencilla.
4. **Escalabilidad**, debido a que permite desplegar múltiples instancias de una aplicación de manera sencilla.

Gracias a la implementación de esta tecnología se ha asegurado que **no surjan errores** en la fase de despliegue debidos a la **incompatibilidad de sistemas operativos** y **obsolescencia o incompatibilidad** entre las **diferentes versiones** de las tecnologías o librerías.

Esto permite que los usuarios que quieran utilizar la aplicación puedan tenerla ejecutándose en su equipo o servidor en cuestión de segundos o minutos.

Para realizar el despliegue de la aplicación los **pasos a seguir** son los siguientes:

1. Clonar el repositorio de GitHub donde se aloja el código de la aplicación.
2. Entrar en la carpeta del proyecto
3. En caso de no tener instalado Docker Desktop, realizar la instalación.
4. Copiar el archivo de ejemplo y configurar las variables de entorno.
5. Usar comandos de Docker para iniciar y desplegar la aplicación.



Figura 44. Ejecución de comandos para el despliegue de la aplicación.

Fuente: Elaboración propia.

Para que el despliegue de la aplicación sea automático y sencillo, ha sido necesario configurar tres archivos dentro del proyecto, con las configuraciones y directivas necesarias para que Docker pudiera crear el contenedor de la aplicación.

La figura muestra el archivo "***docker-compose.yml***", el cual es utilizado para crear los principales servicios de la aplicación. En este archivo se definen configuraciones, volúmenes donde se almacenan datos, nombres y diferentes directivas para cada una de las imágenes de la aplicación:

- **api**, construye la API desde su propio archivo Dockerfile.

```yaml
api:
  build:
    context: ./api
    dockerfile: Dockerfile
  container_name: webscavul_api
  depends_on:
    - mariadb
  restart: always
  env_file: .env
  ports:
    - 8000:80
  volumes:
    - ./api:/webscavul/api
  networks:
    - backend
```

Figura 45. Fragmento del archivo docker-compose.yml para el servicio de la API.

Fuente: Elaboración propia.

- **app**, construye y levanta la interfaz de usuario de Next.js, desde su propio archivo Dockerfile.

```yaml
app:
  build:
    context: ./webapp
    dockerfile: Dockerfile.dev
  container_name: webscavul_app
  depends_on:
    - api
  restart: always
  env_file: .env
  ports:
    - "${APP_PORT}:3000"
  volumes:
    - ./webapp:/webscavul/webapp
    - /webscavul/webapp/node_modules
  networks:
    - backend
```

Figura 46. Fragmento del archivo docker-compose.yml para el servicio de la interfaz.

Fuente: Elaboración propia.

- **mariadb**, descarga la imagen y levanta el servicio de la base de datos de la aplicación.

```yaml
mariadb:
  image: mariadb
  container_name: webscavul_mariadb
  restart: always
  environment:
    MYSQL_DATABASE: '${DB_NAME}'
    MYSQL_ROOT_PASSWORD: "${DB_PASSWORD}"
    MYSQL_USER: "${DB_USER}"
    MYSQL_PASSWORD: "${DB_PASSWORD}"
  volumes:
    - data:/var/lib/mysql
    - ./db_scripts:/docker-entrypoint-initdb.d/:ro
  ports:
    - "${DB_PORT}:3306"
  networks:
    - backend
```

Figura 47. Fragmento del archivo docker-compose.yml para el servicio de la base de datos.

Fuente: Elaboración propia.

A continuación, se muestra el archivo **Dockerfile de la API**, usa una imagen de Python e instala los paquetes y librerías necesarias. El contenedor expone el puerto 80, ejecutando finalmente un comando por consola que inicia el servidor.

```
FROM python:3.12-bookworm

ENV PYTHONUNBUFFERED=1

WORKDIR /webscavul/api

COPY ./requirements.txt .

# Dependencias del sistema y Playwright
RUN apt-get update && apt-get install -y wget gnupg ca-certificates && \
    pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt && \
    playwright install-deps && \
    playwright install

COPY . .

EXPOSE 80

CMD ["uvicorn", "main:api", "--host", "0.0.0.0", "--port", "80", "--reload", "--reload-dir", "."]
```

Figura 48. Código del archivo Dockerfile de la API.

Fuente: Elaboración propia.

Por último, se muestra el archive **Dockerfile de la aplicación de Next.js**, que usa una imagen de Node.js e instala las librerías y dependencias de la aplicación. El contenedor expone el puerto 3000 y ejecuta un comando por consola que levanta esta aplicación.

```
FROM node:22-alpine AS dev

WORKDIR /webscavul/webapp

ENV NODE_ENV=development
ENV NEXT_TELEMETRY_DISABLED=1
ENV CHOKIDAR_USEPOLLING=true
ENV WATCHPACK_POLLING=true
ENV WATCHPACK_POLLING_INTERVAL=300

# Instala dependencias del sistema necesarias
RUN apk add --no-cache libc6-compat

# Copia solo los archivos de dependencias para cachear instalación
COPY package.json yarn.lock* package-lock.json* pnpm-lock.yaml* .npmrc* ./

# Instala dependencias
RUN \
  if [ -f yarn.lock ]; then yarn; \
  elif [ -f package-lock.json ]; then npm install; \
  elif [ -f pnpm-lock.yaml ]; then corepack enable pnpm && pnpm install; \
  else echo "No lockfile found" && exit 1; \
  fi

# Copia el resto del código
COPY . .

EXPOSE 3000

CMD ["npm", "run", "dev"]
```

Figura 49. Código del archivo Dockerfile de la interfaz.

Fuente: Elaboración propia.

4.8. Pruebas y evaluación de la aplicación

En este apartado, se detallan y desarrollan las **pruebas y evaluaciones** realizadas para **validar que los objetivos y requisitos funcionales y técnicos** definidos durante la fase de diseño y planificación de la aplicación han sido cumplidos.

El objetivo principal es demostrar y validar que la aplicación no solo detecta y analiza de forma correcta las vulnerabilidades para las que ha sido diseñada y programada, sino que también ofrece a los usuarios un experiencia accesible, fluida e intuitiva.

Para esto, se realizarán dos tipos de evaluaciones:

1. **Pruebas de usabilidad y experiencia de usuario (UX/UI)**, tienen el fin de analizar la interacción del usuario con la interfaz diseñada, su experiencia y la utilidad y claridad de la información presentada.
2. **Pruebas de funcionamiento**, tienen el fin de verificar la correcta ejecución de las funcionalidades implementadas en la aplicación.

4.8.1. Pruebas de usabilidad y experiencia de usuario

La finalidad de las pruebas de usabilidad y experiencia de usuario es **evaluar la forma en la que las personas interactúan con la aplicación**, para conocer cuál es la opinión de estos sobre la accesibilidad, eficiencia, utilidad y claridad de la información y satisfacción general. Proporcionando de este modo una base objetiva que sirve para valorar la calidad de la aplicación desarrollada desde la perspectiva de los usuarios y **orientar posibles mejoras futura**.

La encuesta fue llevada a cabo mediante la herramienta Google Forms y en el cuestionario se combinaron preguntas basadas en el **System Usability Scale (SUS)** (Busquets, 2025), además de otras de **elaboración propia**.

Fue respondida por **10 participantes**, 5 de ellos son desarrolladores web o con experiencia en desarrollo web o ciberseguridad y los otros 5 participantes no tienen ningún tipo de experiencia en estos ámbitos, con el fin de conocer la opinión de diferentes perfiles de usuario.

A continuación, se muestra una tabla con las puntuaciones de los usuarios encuestados, con las medias de cada pregunta, agrupando los usuarios según su nivel de experiencia, además de una media global de todos ellos.

Tabla 3. Resultado de la encuesta sobre la usabilidad y experiencia de usuario.

Fuente: Elaboración propia.

Participante	Experiencia	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10
1	No	5	3	4	2	5	5	1	5	5	5
2	No	5	3	4	2	5	5	1	4	5	5
3	No	4	3	4	3	5	5	2	4	4	5
4	No	4	4	4	2	5	5	2	4	4	5
5	No	4	3	4	3	4	3	2	4	4	3
Valor medio, sin experiencia		4,4	3,2	4	2,4	4,8	4,6	1,6	4,2	4,4	4,6
6	Si	4	2	5	2	5	5	2	5	5	4
7	Si	4	1	4	3	4	4	2	4	4	5
8	Si	5	2	3	2	5	5	1	4	4	5
9	Si	5	1	4	1	5	4	1	5	5	5
10	Si	5	1	5	2	5	5	1	5	4	5
Valor medio, con experiencia		4,6	1,4	4,2	2	4,8	4,6	1,4	4,6	4,4	4,8
Valor medio total		4,5	2,3	4,1	2,2	4,8	4,6	1,4	4,4	4,4	4,7

Para realizar el análisis de los datos obtenido se van a desglosar estos por cada pregunta:

1. **Facilidad de uso**, la puntuación media de esta pregunta es de 4,5 sobre 5, lo que indica que los participantes consideran que la aplicación es fácil de usar, coincidiendo en este punto los dos grupos de usuarios.
2. **Necesidad de apoyo técnico**, la puntuación media es de 2,3 aunque, es importante en este caso, el resultado del grupo de usuario con experiencia (1,4) ya que estos son realmente el público objetivo de la aplicación, indicando que no es necesario la ayuda técnica para realizar el escaneo y análisis.
3. **Aprendizaje**, la puntuación media es de 4,1 lo que indica que los usuarios consideran que la aplicación tiene un diseño y flujo de trabajo intuitivo y fácil de seguir.
4. **Complejidad inicial**, la puntuación media es de 2,2 siendo muy parecida entre los dos grupos, esto indica que no existe una gran barrera de entrada, en término de conocimientos técnicos, para poder usar la aplicación.
5. **Rapidez y eficiencia del escaneo**, la media total es de 4,8 coincidiendo ambos grupos, siendo este un indicador de que los usuarios perciben los tiempos de ejecución como eficientes.
6. **Navegación fluida**, la media total es de 4,6 coincidiendo de nuevo ambos grupos, indicando que la interfaz de la aplicación está bien diseñada.
7. **Claridad de la información**, la media total es de 1,4 con una puntuación similar ambos grupos, indicando que la forma en la que se presenta la información y se explican los conceptos es clara y entendible.
8. **Utilidad de la información**, la media total es de 4,4 indicando que la información que genera la aplicación es perciba como útil para el análisis de vulnerabilidades, confirmando así que se cumple con objetivo principal de la aplicación.
9. **Coherencia de los elementos de la aplicación**, la media total es de 4,4 indicando que la interfaz de la aplicación cuenta con los elementos necesarios para el desarrollo de la tarea sin distraer al usuario.
10. **Diseño**, con un valor medio total de 4,7 indica que los usuarios perciben el diseño de la aplicación profesional y visualmente agradable.

Tras el análisis realizado se puede concluir que la aplicación cumple con los objetivos de usabilidad y accesibilidad que se buscaban, siendo las **fortalezas** de esta:

1. Facilidad de uso y aprendizaje.
2. Rapidez y eficiencia del escaneo.
3. Diseño, accesibilidad y coherencia de la interfaz.

Con el fin de dar contexto e información, se adjunta una copia de las preguntas del cuestionario realizado en el Anexo A.

4.8.2. Pruebas de funcionamiento

La finalidad de las pruebas de funcionamiento es la verificación de que la herramienta detecta, analiza e identifica de forma correcta las vulnerabilidades presentes en las páginas web.

La metodología usada consiste en la creación de un **archivo HTML de prueba, simulando una página web real**, que contiene todos los tipos de vulnerabilidades que la herramienta debería detectar, así como casos en los que esta no debería alertar o avisar al usuario.

Estas **vulnerabilidades** han sido **colocadas de forma deliberada** en elementos como formulario, etiquetas de estilos, recursos externos, enlaces… siendo un total de 34 vulnerabilidades las que la herramienta debe detectar.

A continuación, se presentan estas vulnerabilidades desglosadas por tipo de elemento en el que están presentes.

4.8.2.1. Vulnerabilidades en meta etiquetas

Tabla 4. Listado de vulnerabilidades encontradas por Webscavul en meta etiquetas. Fuente: Elaboración propia.

No	Vulnerabilidad	Importancia
1	Meta tag CSP no configurada.	Alto
2	Meta tag refresh con valor de redirección inseguro (HTTP), con valor *http://test-site.com.*	Alto

3	Meta tag referrer con valor no recomendado, con valor *unsafe-url.*	Medio
4	Meta tag generator encontrada, con valor *WordPress 6.0.*	Informativo

4.8.2.2. Vulnerabilidades en etiquetas script

Tabla 5. Listado de vulnerabilidades encontradas por Webscavul en etiquetas script.

Fuente: Elaboración propia.

No	Vulnerabilidad	Importancia
5	Script externo inseguro (HTTP), con valor *http://insecure-cdn.com/script.js.*	Bajo
6	Script sin crossorigin ni integrity, en el recurso *http://insecure-cdn.com/script.js.*	Medio
7	Script sin crossorigin ni integrity, en el recurso *https://external-cdn.com/script.js.*	Medio
8	Script sin crossorigin ni integrity, en el recurso *https://external-cdn.com/script2.js.*	Medio
9	Script con integrity sin crossorigin, en el recurso *https://external-cdn.com/script3.js.*	Medio

4.8.2.3. Vulnerabilidades en etiquetas link

Tabla 6. Listado de vulnerabilidades encontradas por Webscavul en etiquetas link.

Fuente: Elaboración propia.

No	Vulnerabilidad	Importancia
10	Link sin crossorigin ni integrity, en el recurso *https://cdn.jsdelivr.net/npm/bootstrap...*	Bajo

11	Link externo inseguro (HTTP), en el recurso *http://insecure-cdn.com/style.css.*	Bajo
12	Link sin crossorigin ni integrity, en el recurso *http://insecure-cdn.com/style.css.*	Bajo
13	Link sin crossorigin ni integrity, en el recurso *https://external-cdn.com/style.css.*	Bajo
14	Link con crossorigin sin integrity, en el recurso *https://external-cdn.com/style2.css*.	Bajo
15	Link con integrity sin crossorigin, en el recurso *https://external-cdn.com/style3.css.*	Bajo
16	Link con atributos charset, rev deprecados	Medio

4.8.2.4. Vulnerabilidades en formularios

Tabla 7. Listado de vulnerabilidades encontradas por Webscavul en formularios. Fuente: Elaboración propia.

No	Vulnerabilidad	Importancia
17	Formulario sin token CSRF implementado, con el identificador *form-vulnerable-1*.	Informativo
18	Formulario con atributo “action” externo inseguro (HTTP), con el identificador *form-vulnerable-2* y valor de “action” *http://test-site.com/process*.	Alto
19	Formulario con datos confidenciales expuestos (GET), con el identificador *form-vulnerable-3*.	Alto

20	Formulario con atributo “action” vacío, con el identificador *form-vulnerable-4*.	Informativo
21	Formulario con campo de contraseña sin usar tipo de input correcto (password), con el identificador *form-vulnerable-5* y tipo “text”	Medio
22	Formulario con campo de contraseña sin usar tipo de input correcto (password), con el identificador *form-vulnerable-6* y tipo “textarea”	Medio
23	Formulario sin token CSRF implementado, con el identificador *form-login-vulnerable* en tipo de formulario crítico (login)	Alto
24	Formulario sin token CSRF implementado, con el identificador *multi-vulnerable-form*.	Informativo
25	Formulario con atributo “action” externo inseguro (HTTP), con el identificador *multi-vulnerable-form* y valor de “action” *http://bad-site.com*.	Alto
26	Formulario con campo de contraseña sin usar tipo de input correcto (password), con el identificador *multi-vulnerable-form* y tipo “text”.	Medio
27	Formulario con datos confidenciales expuestos (GET), con el identificador *multi-vulnerable-form*.	Alto
28	Formulario con campo de contraseña sin usar tipo de input correcto (password), con el identificador *multi-vulnerable-form*.	Medio
29	Formulario con datos confidenciales expuestos (GET), con el identificador *multi-vulnerable-form*.	Alto

30	Formulario con atributo “action” vacío, con el identificador *empty-action-form*.	Informativo

4.8.2.5. Vulnerabilidades en enlaces

Tabla 8. Listado de vulnerabilidades encontradas por Webscavul en enlaces. Fuente: Elaboración propia.

No	Vulnerabilidad	Importancia
31	Enlace externo inseguro (HTTP), con valor *http://insecure-site.com*.	Bajo
32	Enlace con target _blank sin rel=noopener noreferrer, con valor *https://external-site.com*.	Medio
33	Enlace con target _blank sin rel=noopener, con valor *https://external-site.com*.	Medio
34	Enlace con target _blank sin rel=noreferrer, con valor *https://external-site.com*.	Medio

Como se he podido observar la herramienta ha demostrado ser capaz de detectar e identificar todas las vulnerabilidades y casos para los que ha sido diseñado. Además, ha conseguido detectar los casos en los que un mismo elemento presentaba varias vulnerabilidades y ha obviado los elementos que no presentan ninguna vulnerabilidad.

Estos resultados ponen de manifiesto que la herramienta desarrollada en este trabajo resulta efectiva y cumple de forma adecuada con el propósito para el cual fue creada.

Con el fin de aportar transparencia y reproducibilidad, se adjunta el código HTML usado en las pruebas de funcionamiento en el Anexo B.

5. Conclusiones

Webscavul, se planteó como una herramienta que lograra facilitar el análisis y gestión de vulnerabilidades en páginas web, contara con una interfaz clara y fácil de usar y fuera gratuita y de código abierto, con el fin de romper la barrera técnica y la necesidad de contar con conocimientos amplios en materia de ciberseguridad que muchas herramientas actuales demandan, contribuyendo así a mejorar la seguridad en el desarrollo de aplicaciones web y la gestión eficiente de los recursos.

A lo largo de este apartado, se expondrán los objetivos definidos en este trabajo, así como los resultados y aportaciones derivados del desarrollo de la herramienta, con el fin de analizar el grado de cumplimiento obtenido, la importancia y calidad de la solución desarrollada, así como su relevancia.

El objetivo general de este trabajo consistía en el desarrollo de una aplicación que permitiera **detectar vulnerabilidades y fallos presentes en páginas web**, con una **interfaz clara**, accesible y fácil de usar, que incluyera una **herramienta nativa para la gestión** de los fallos encontrados y que ofreciera un escaneo automático y eficiente. En vista de los resultados obtenidos, tanto en las pruebas realizadas, como en la calidad y funcionamiento general de la aplicación, puede concluirse que el objetivo general planteado se cumple de forma satisfactoria, integrando Webscavul todas las características citadas anteriormente.

Además, debía ser una solución de **gratuita y de código abierto**, que se enfocara en eliminar la barrera de entrada para los usuarios sin amplios conocimientos técnicos, usando tecnologías y lenguajes actuales y modernos. Webscavul cumple con este objetivo al ofrecer el desarrollo de una aplicación *full-stack,* que combina un motor de escaneo automático y eficiente con una interfaz que enfoca y orienta su diseño a la usabilidad y experiencia de usuario, alojada en un repositorio público de código, donde los usuarios interesados, no solo pueden utilizar la aplicación de forma gratuita, si no que pueden adaptar esta a sus necesidades o requerimientos y realizar aportaciones que ayuden a corregir errores o incluir más funcionalidades y características a la herramienta.

Dentro de los objetivos específicos definidos, el primero de ellos planteaba la investigación de las principales vulnerabilidades presente en las aplicaciones web modernas, siguiendo estándares reconocidos como OWASP Top Ten. Este objetivo ha sido alcanzado mediante el

estudio y análisis de las diferentes vulnerabilidades, lo que ha permitido que el desarrollo de Webscavul haya contado con una base sólida de conocimiento orientándose así de la mejor forma posible.

En segundo lugar, se planteaba el **análisis y prueba de funcionamiento de las diferentes herramientas y soluciones actuales**. Este análisis ha ayudado a detectar y comprender las diferentes limitaciones que tienen estas herramientas, sirviendo esto como punto de referencia a la hora de definir y diseñar las funcionalidad o características de Webscavul, concluyendo que la herramienta logra ser una alternativa o complemento a estas soluciones.

El tercer objetivo buscaba el diseño y desarrollo de una **herramienta que contara con una interfaz amigable y sencilla** para los usuarios y que mostrara la información y los resultados obtenidos de una forma clara y entendible. Webscavul cumple satisfactoriamente con este objetivo, tal y como se ha demostrado en las pruebas usabilidad y experiencia de usuario realizadas a 10 usuarios que probaron la herramienta. En estas pruebas realizadas se puede concluir que las fortalezas o puntos fuertes de la herramienta es que se percibe como intuitiva, eficiente, accesible, útil y fácil de usar.

El cuarto objetivo consistía en la **investigación e implementación de modelos de lenguaje natural** que permitieran a la herramienta generar reportes con información menos técnica y por lo tanto más accesible y entendible para los usuarios. Este objetivo se cumple de igual forma, debido a que Webscavul integra de forma nativa, modelos de lenguaje que generan soluciones y contexto a las vulnerabilidades detectadas por la aplicación, reduciendo así la necesidad de conocimientos técnicos.

Por último, el quinto objetivo consistía en la **evaluación de la eficacia y funcionamiento general de la aplicación,** así como de cada una de las funcionalidades desarrolladas e implementadas en esta, usando para ello, entornos de pruebas controlados, que permitan la obtención de resultados objetivos, comparables y repetibles. Para ello, a lo largo del desarrollo, se realizaron pruebas de funcionamiento continuas, para cada una de las nuevas funcionalidades tras su implementación. Además, se realizó una prueba final que simulaba un entorno real, con vulnerabilidades y fallos introducidos de manera deliberada, que ha ayudado a concluir que la herramienta es capaz de identificar las vulnerabilidades de forma correcta, demostrando así su efectividad y eficiencia.

Es por esto, que los objetivos propuestos a lo largo de este trabajo, tanto generales como específicos, se han cumplido de forma satisfactoria, demostrando que Webscavul funciona de forma correcta, cumple los requisitos de funcionamiento y diseño planteados y que es capaz de detectar, analizar y gestionar las vulnerabilidades encontradas en páginas web.

En conclusión, Webscavul es una herramienta que constituye una solución tanto complementaria a otras herramientas como una opción gratuita, robusta y fácil de usar que ayuda a mejorar la seguridad en el desarrollo de aplicaciones web.

6. Trabajo futuro

El proyecto actual cuenta con una base sólida, pero debido al tiempo disponible para su desarrollo, hay numerosas funcionalidades o mejoras, las cuales, no ha sido posible incluir y que aportarían valor significativo a la aplicación.

En este apartado se presentan las líneas de trabajo futuro:

1. Detección y pruebas de ataques inyecciones SQL y XSS en formularios, para ello se debería desarrollar un módulo *fuzzer* que insertara los posibles *payloads* y comprobara si ha tenido éxito analizando la respuesta del servidor.
2. Detección de los métodos HTTP incorporados por el servidor, para detectar posibles configuraciones inseguras.
3. Añadir la posibilidad de programar y automatizar escaneos, haciendo que le proceso de escaneo fuera periódico y eficiente sin necesidad de realizar de manera manual.
4. Integración de librerías para la detección de tecnologías utilizadas en la aplicación, con el fin de añadir más información de vulnerabilidades conocidas (CVE) al informe generado por la herramienta.
5. Añadir una herramienta de Análisis de Composición de Software, que fuera capaz de detectar archivos con las versiones de la librería, o daría la posibilidad al usuario de adjuntar el archivo (*requirementes.txt, package.json*), con el fin de generar un informe de vulnerabilidades y versiones recomendadas.
6. Búsqueda de rutas, archivos o directorios ocultos para detectar posible información que podría ser sensible o de acceso no autorizado.
7. Añadir funcionalidades para la creación de cuentas de usuarios y equipos, lo cual daría la posibilidad de desplegar la aplicación como servicio (SaaS), no solo siendo una herramienta local, si no una plataforma multiusuario.

Referencias bibliográficas

Statista. (2024). *Number of internet users worldwide from 2005 to 2024 (in millions)*. Obtenido de Statista: https://www.statista.com/statistics/273018/number-of-internet-users-worldwide/

Netcraft. (Marzo de 2025). *March 2025 Web Server Survey*. Obtenido de Netcraft: https://www.netcraft.com/blog/march-2025-web-server-survey/

Foundation, O. (2021). *OWASP Top Ten*. Obtenido de OWASP: https://owasp.org/www-project-top-ten/

Bach-Nutman, M. (2020). Understanding The Top 10 OWASP Vulnerabilities. *arXiv*.

ENISA, E. U. (2024). *ENISA threat landscape 2024.* Atenas: ENISA. Obtenido de https://www.enisa.europa.eu/sites/default/files/2024-11/ENISA%20Threat%20Landscape%202024_0.pdf

Cloudflare. (s.f.). *¿Qué es OWASP? ¿Qué es el OWASP Top 10?* Obtenido de Cloudflare: https://www.cloudflare.com/es-es/learning/security/threats/owasp-top-10/

Zamora, D. B. (June de 2024). *Evaluación de vulnerabilidades en aplicaciones web: un enfoque OWASP Top 10.* Obtenido de https://accedacris.ulpgc.es/bitstream/10553/130999/1/TFG%20Betancor%20Zamora.pdf

Richard Amankwah, J. C. (2020). An empirical comparison of commercial and open-source web vulnerability scanners. *Academia.edu*.

Swetha B., S. S. (2024). A Comparative Analysis of Vulnerability Management Tools: Evaluating Nessus, Acunetix, and Nikto for Risk-Based Security Solutions. *ResearchGate*.

PortSwigger. (2025). *Burp Suite tools* . Retrieved from PortSwigger: https://portswigger.net/burp/documentation/desktop/tools

Muzun Althunayyan, N. S. (2022). Evaluation of Black-Box Web Application Security Scanners in Detecting Injection Vulnerabilities. *MDPI*.

Jahanzeb Shahid, M. K. (2022). A Comparative Study of Web Application Security Parameters: Current Trends and Future Directions . *MDPI*.

Cavallo, S. (2024). *La importancia de una interfaz de usuario eficaz.* Obtenido de Initium: https://www.initiumsoft.com/blog_initium/interfaz-de-usuario/

Firch, J. (2024). *Why Vulnerability Assessment Reports Fail (& How To Fix It)*. Obtenido de PurpleSec: https://purplesec.us/learn/vulnerability-assessment-reporting

Snyk. (2023). *2023 State of Open Source Security Repor.* Snyk.

FUENTES DEL BURGO, J., & SEBASTIÁN PÉREZ, M. Á. (2022). Análisis comparativo de la herramienta tablero en las metodologías ágiles Scrum, Kanban y Scrumban en proyectos de software. *Aeipro*.

Corporation, M. (25 de Junio de 2020). *How to connect Python programs to MariaDB*. Obtenido de MariaDB: https://mariadb.com/resources/blog/how-to-connect-python-programs-to-mariadb/

FastAPI. (s.f.). *FastAPI*. Obtenido de FastAPI Documentation: https://fastapi.tiangolo.com

Reitz, K. (2025). *Requests Documentation*. Obtenido de Requests: HTTP for Humans: https://requests.readthedocs.io/en/latest/

Mukhamadiev, A. (2018). *Transitioning from server-side to client-side.* Helsinki: Metropolia University of Applied Sciences.

Richardson, L. (2025). *Documentación de Beautiful Soup*. Obtenido de Crummy: https://www.crummy.com/software/BeautifulSoup/bs4/doc.es/

pyOpenSSL. (2025). *Welcome to pyOpenSSL's documentation!* Obtenido de pyOpenSSL Documentation: https://www.pyopenssl.org/en/latest/

Vercel. (2025). *Next.js Documentation*. Obtenido de Next.js: https://nextjs.org/docs

Nalpas, M. (30 de Julio de 2020). *Prácticas recomendadas de Referrer-Policy y Refere*. Obtenido de web.dev.

Subresource Integrity. (s.f.). Obtenido de MDN: https://developer.mozilla.org/en-US/docs/Web/Security/Subresource_Integrity

MDN. (2025). *Content Security Policy (CSP)*. Obtenido de MDN Web Docs: https://developer.mozilla.org/es/docs/Web/HTTP/Guides/CSP

MDN. (2025). *Strict-Transport-Security*. Obtenido de MDN Web Docs: https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Headers/Strict-Transport-Security

MDN. (2025). *X-Frame-Options*. Obtenido de MDN Web Docs: https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Headers/X-Frame-Options

MDN. (2025). *X-Content-Type-Options*. Obtenido de MDN Web Docs: https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Headers/X-Content-Type-Options

MDN. (2025). *Set-Cookie*. Obtenido de MDN Web Docs: https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Headers/Set-Cookie

MDN. (2025). *Cache-Control*. Obtenido de MDN Web Docs: https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Headers/Cache-Control

MDN. (2025). *X-XSS-Protection*. Obtenido de MDN Web Docs: https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Headers/X-XSS-Protection

MDN. (2025). *Referrer-Policy*. Obtenido de MDN Web Docs: https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Headers/Referrer-Policy

MDN. (2025). *Permissions-Policy header*. Obtenido de MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Permissions-Policy

MDN. (2025). *Refresh header*. Obtenido de MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Refresh

Busquets, C. (2025). *Medir la usabilidad con el Sistema de Escalas de Usabilidad (SUS)*. Obtenido de uifrommars: https://www.uifrommars.com/como-medir-usabilidad-que-es-sus/

Anexo A. Encuesta de usabilidad y experiencia de usuario

P0. Antes de empezar el cuestionario, ¿trabaja o tiene experiencia en desarrollo web o ciberseguridad?

Por favor, indica tu nivel de acuerdo o desacuerdo con las siguientes afirmaciones basándote en tu experiencia usando la herramienta Webscavul.

P1. Creo que la aplicación es sencilla y fácil de usar.

P2. Necesito apoyo técnico para utilizar esta aplicación.

P3. Creo que la mayoría de la gente puede aprender a utilizar la aplicación rápidamente.

P4. Creo que hay muchas cosas que aprender antes de poder empezar a utilizar la aplicación.

P5. Creo que el tiempo que la aplicación emplea en realizar el escaneo es rápido y eficiente.

P6. La experiencia de navegación por la aplicación se sintió fluida y sin interrupciones durante su uso.

P7. La información que presenta es complicada o confusa y hay muchas partes que no logro entender.

P8. Creo que la información proporcionada por la aplicación es valiosa para la tarea a desempeñar.

P9. Todos los elementos presentes en la aplicación son útiles y necesarios

P10. El diseño de la aplicación me parece profesional y visualmente agradable.

Anexo B. Archivo HTML de pruebas

```
<!DOCTYPE html>
<html lang="es">

<head>
    <!-- CORRECTO: charset utf-8 configurado -->
    <meta charset="utf-8">

    <!-- DEBE DETECTAR: Meta tag generator encontrada -->
    <meta name="generator" content="WordPress 6.0">

    <!-- DEBE DETECTAR: Meta tag referrer con valor no recomendado -->
    <meta name="referrer" content="unsafe-url">

    <!-- CORRECTO: Meta robots configurada -->
    <meta name="robots" content="index, follow">

    <!-- DEBE DETECTAR: Meta tag CSP no configurada - si no hay header CSP -->
    <!-- Si hay CSP por header, esto no debería detectarse -->

    <!-- DEBE DETECTAR: Meta tag refresh con redirección insegura -->
    <meta http-equiv="refresh" content="300;url=http://test-site.com">

    <!-- CORRECTO: Link con HTTPS -->
    <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css">

    <!-- DEBE DETECTAR: Link externo inseguro HTTP -->
    <link rel="stylesheet" href="http://insecure-cdn.com/style.css">
```

```
    <!-- DEBE DETECTAR: Link sin crossorigin ni integrity -->
    <link rel="stylesheet" href="https://external-cdn.com/style.css">

    <!-- DEBE DETECTAR: Link con crossorigin sin integrity -->
    <link rel="stylesheet" href="https://external-cdn.com/style2.css"
crossorigin="anonymous">

    <!-- DEBE DETECTAR: Link con integrity sin crossorigin -->
    <link rel="stylesheet" href="https://external-cdn.com/style3.css"
integrity="sha384-example">

    <!-- CORRECTO: Link con crossorigin e integrity configurados -->
    <link rel="stylesheet" href="https://external-cdn.com/style4.css"
crossorigin="anonymous"
        integrity="sha384-
1BmE4kWBq78iYhFldvKuhfTAU6auU8tT94WrHftjDbrCEXSU1oBoqyl2QvZ6jIW3">

    <!-- DEBE DETECTAR: Link con atributos deprecados -->
    <link rel="stylesheet" href="style.css" charset="utf-8" rev="stylesheet">

    <!-- DEBE DETECTAR: Script externo inseguro HTTP -->
    <script src="http://insecure-cdn.com/script.js"></script>

    <!-- DEBE DETECTAR: Script sin crossorigin ni integrity -->
    <script src="https://external-cdn.com/script.js"></script>

    <!-- DEBE DETECTAR: Script con crossorigin sin integrity -->
    <script src="https://external-cdn.com/script2.js"
crossorigin="anonymous"></script>
```

```
    <!-- DEBE DETECTAR: Script con integrity sin crossorigin -->
    <script src="https://external-cdn.com/script3.js" integrity="sha384-
example"></script>

    <!-- CORRECTO: Script con crossorigin e integrity configurados -->
    <script src="https://external-cdn.com/script4.js" crossorigin="anonymous"
        integrity="sha384-
1BmE4kWBq78iYhFldvKuhfTAU6auU8tT94WrHftjDbrCEXSU1oBoqyl2QvZ6jIW3"></script>

    <!-- CORRECTO: Script local sin problemas -->
    <script src="js/local-script.js"></script>

    <title>Página de Prueba de Vulnerabilidades</title>
</head>

<body>
    <h1>Página de Prueba para Detección de Vulnerabilidades</h1>

    <h2>Vulnerabilidades en Scripts</h2>
    <p>Ya incluidos en el head de la página</p>

    <h2>Vulnerabilidades en Scripts</h2>
    <p>Ya incluidos en el head de la página</p>

    <h2>Vulnerabilidades en Formularios</h2>

    <!-- DEBE DETECTAR: Formulario sin token CSRF -->
    <form id="form-vulnerable-1" action="process.php" method="post">
        <label for="username">Usuario:</label>
        <input type="text" id="username" name="username">
```

```
        <label for="password">Contraseña:</label>
        <input type="password" id="password" name="password">
        <button type="submit">Enviar</button>
    </form>

    <!-- DEBE DETECTAR: Formulario con action HTTP externo -->
    <form id="form-vulnerable-2" action="http://test-site.com/process"
method="post">
        <input type="hidden" name="csrf_token" value="abc123">
        <label for="data">Datos:</label>
        <input type="text" id="data" name="data">
        <button type="submit">Enviar</button>
    </form>

    <!-- DEBE DETECTAR: Formulario con datos confidenciales en GET -->
    <form id="form-vulnerable-3" action="login.php" method="get">
        <input type="hidden" name="csrf_token" value="abc123">
        <label for="user">Usuario:</label>
        <input type="text" id="user" name="user">
        <label for="pass">Contraseña:</label>
        <input type="password" id="pass" name="password">
        <button type="submit">Login</button>
    </form>

    <!-- DEBE DETECTAR: Formulario sin action -->
    <form id="form-vulnerable-4" method="post">
        <input type="hidden" name="csrf_token" value="abc123">
        <label for="comment">Comentario:</label>
        <textarea id="comment" name="comment"></textarea>
        <button type="submit">Comentar</button>
```

```
</form>

<!-- DEBE DETECTAR: Campo password con tipo text -->
<form id="form-vulnerable-5" action="change-password.php" method="post">
    <input type="hidden" name="csrf_token" value="abc123">
    <label for="old-pwd">Contraseña actual:</label>
    <input type="text" id="old-pwd" name="password_old">
    <label for="new-pwd">Nueva contraseña:</label>
    <input type="password" id="new-pwd" name="password_new">
    <button type="submit">Cambiar</button>
</form>

<!-- DEBE DETECTAR: Campo password en textarea -->
<form id="form-vulnerable-6" action="process.php" method="post">
    <input type="hidden" name="csrf_token" value="abc123">
    <label for="pwd-area">Contraseña:</label>
    <textarea id="pwd-area" name="password"></textarea>
    <button type="submit">Enviar</button>
</form>

<!-- DEBE DETECTAR: Formulario login sin CSRF -->
<form id="form-login-vulnerable" action="login.php" method="post">
    <label for="login-user">Usuario:</label>
    <input type="text" id="login-user" name="username">
    <label for="login-pass">Contraseña:</label>
    <input type="password" id="login-pass" name="password">
    <button type="submit">Iniciar Sesión</button>
</form>

<!-- CORRECTO: Formulario con CSRF token -->
```

```
    <form id="form-correct-1" action="https://secure-site.com/process"
method="post">
        <input type="hidden" name="csrf_token" value="secure-token-123">
        <label for="safe-data">Datos seguros:</label>
        <input type="text" id="safe-data" name="data">
        <button type="submit">Enviar Seguro</button>
    </form>

    <!-- CORRECTO: Formulario con captcha -->
    <form id="form-correct-2" action="submit.php" method="post">
        <div class="g-recaptcha" data-sitekey="your-site-key"></div>
        <label for="message">Mensaje:</label>
        <textarea id="message" name="message"></textarea>
        <button type="submit">Enviar</button>
    </form>

    <h2>Vulnerabilidades en Enlaces</h2>

    <!-- DEBE DETECTAR: Enlace externo inseguro HTTP -->
    <a href="http://insecure-site.com" id="link-vulnerable-1">Sitio Inseguro</a>

    <!-- DEBE DETECTAR: Enlace con target _blank sin rel -->
    <a href="https://external-site.com" target="_blank" id="link-vulnerable-
2">Abrir en nueva ventana sin protección</a>

    <!-- DEBE DETECTAR: Enlace con target _blank sin noopener -->
    <a href="https://external-site.com" target="_blank" rel="noreferrer" id="link-
vulnerable-3">Solo noreferrer</a>

    <!-- DEBE DETECTAR: Enlace con target _blank sin noreferrer -->
```

```
    <a href="https://external-site.com" target="_blank" rel="noopener" id="link-
vulnerable-4">Solo noopener</a>

    <!-- CORRECTO: Enlace interno -->
    <a href="about.html" id="link-correct-1">Página Interna</a>

    <!-- CORRECTO: Enlace externo seguro -->
    <a href="https://secure-external-site.com" id="link-correct-2">Sitio Externo
Seguro</a>

    <!-- CORRECTO: Enlace con target _blank y rel completo -->
    <a href="https://external-site.com" target="_blank" rel="noopener noreferrer"
id="link-correct-3">Enlace Seguro
        Externo</a>

    <!-- CORRECTO: Enlace sin target _blank -->
    <a href="https://external-site.com" id="link-correct-4">Enlace sin target
blank</a>

    <h2>Casos Especiales y Pruebas Adicionales</h2>

    <!-- Formulario con múltiples vulnerabilidades -->
    <form id="multi-vulnerable-form" action="http://bad-site.com" method="get">
        <label for="pwd-text">Password como text:</label>
        <input type="text" id="pwd-text" name="contrasena">
        <label for="pwd-textarea">Password en textarea:</label>
        <textarea id="pwd-textarea" name="pwd"></textarea>
        <button type="submit">Múltiples Vulnerabilidades</button>
    </form>

    <!-- Formulario vacío para probar action vacío -->
```

```
    <form id="empty-action-form" action="" method="post">
        <input type="hidden" name="csrf_token" value="token">
        <input type="text" name="test">
        <button type="submit">Action Vacío</button>
    </form>

    <!-- Enlaces con diferentes casos -->
    <div>
        <a href="mailto:test@example.com">Email Link</a> <!-- NO debe detectar -->
        <a href="javascript:void(0)">JavaScript Link</a> <!-- NO debe detectar -->
        <a href="#section">Anchor Link</a> <!-- NO debe detectar -->
        <a href="tel:+1234567890">Phone Link</a> <!-- NO debe detectar -->
    </div>
</body>

</html>
```